



汇编语言与微机接口技术

第3章 指令系统（二）

2025年2月10日



信息与软件工程学院
School of Information and Software Engineering

3.4

逻辑运算和移位指令

3.4 逻辑运算和移位指令

指令类型



3.4 逻辑运算和移位指令

» 逻辑运算



逻辑运算指令对操作数的要求大多与MOV指令相同。

“非”运算指令要求操作数不能是立即数, 且是单地址操作数;

除“非”运算指令外, 其余指令的执行都会使标志位 $OF=CF=0$

3.4 逻辑运算和移位指令

» “与”指令



格式

AND OPRD1, OPRD2



操作

两操作数按位相“与”，结果送目标地址。

3.4 逻辑运算和移位指令

» “与”指令的应用

应用

01

实现两操作数按位相与的运算

- `AND BL, [SI]`

02

使目标操作数的某些位不变，某些位清（置）零

- `AND AL, 0FH`

高4位清零，低4位不变

03

在操作数不变的情况下使CF和OF清零

- `AND AX, AX`

OR、XOR都有此特性

3.4 逻辑运算和移位指令

» “与”指令应用例

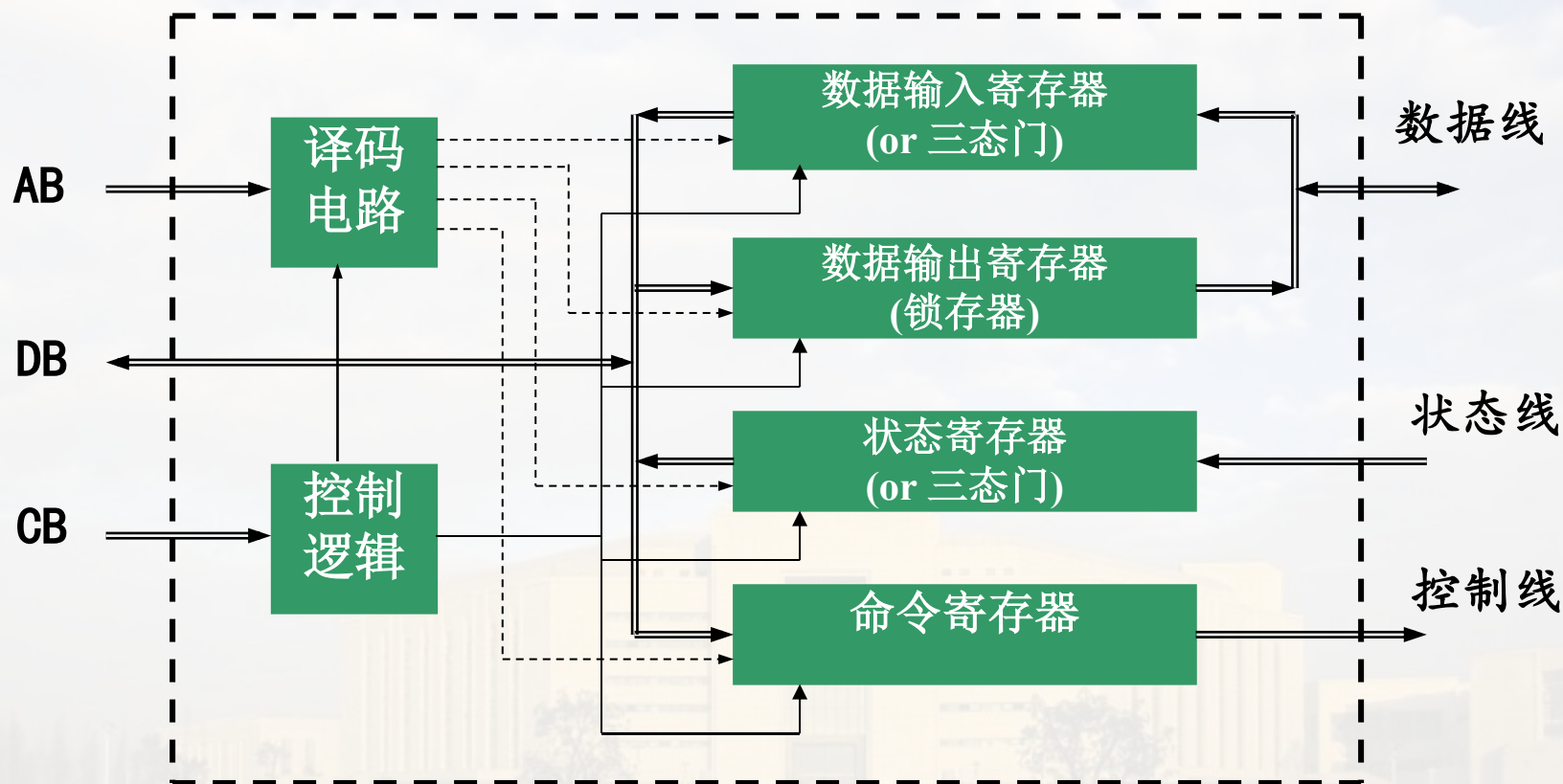
检查（打印机）端口与计算机是否连接正常？

从地址为3F8H端口（状态端口）中读入一个字节数，如果该数bit1位为1，则将内存中DATA为首地址的一个字输出到500FH端口（数据端口），否则就不能进行数据传送。

编写实现该功能的程序段。

3.4 逻辑运算和移位指令

» 接口的基本构成



3.4 逻辑运算和移位指令

» I/O系统中的接口和端口的地址

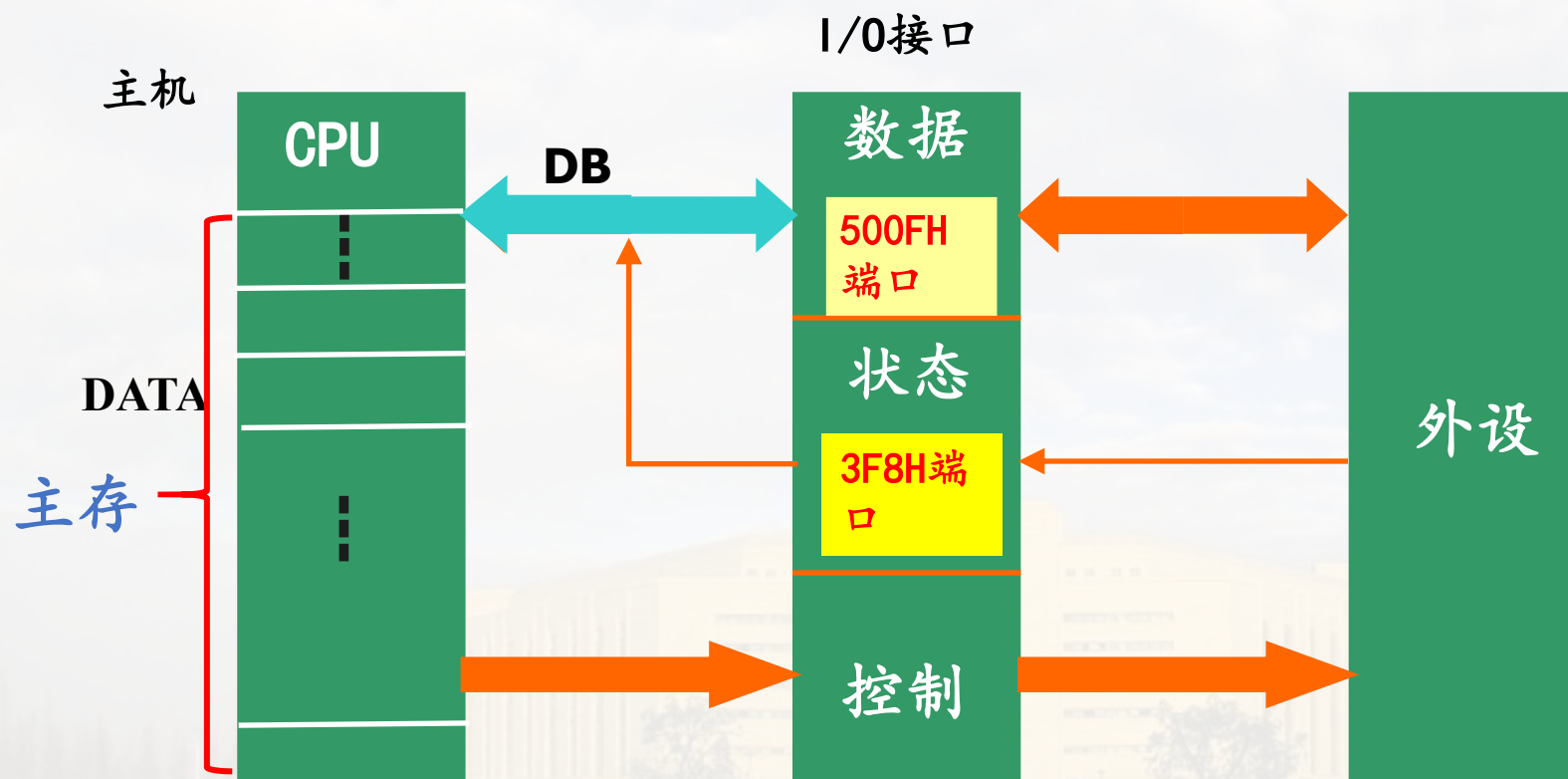


端口地址=芯片地址（高位地址）+片内地址

类似于学号=学院(专业)编号+班号+班内编号

3.4 逻辑运算和移位指令

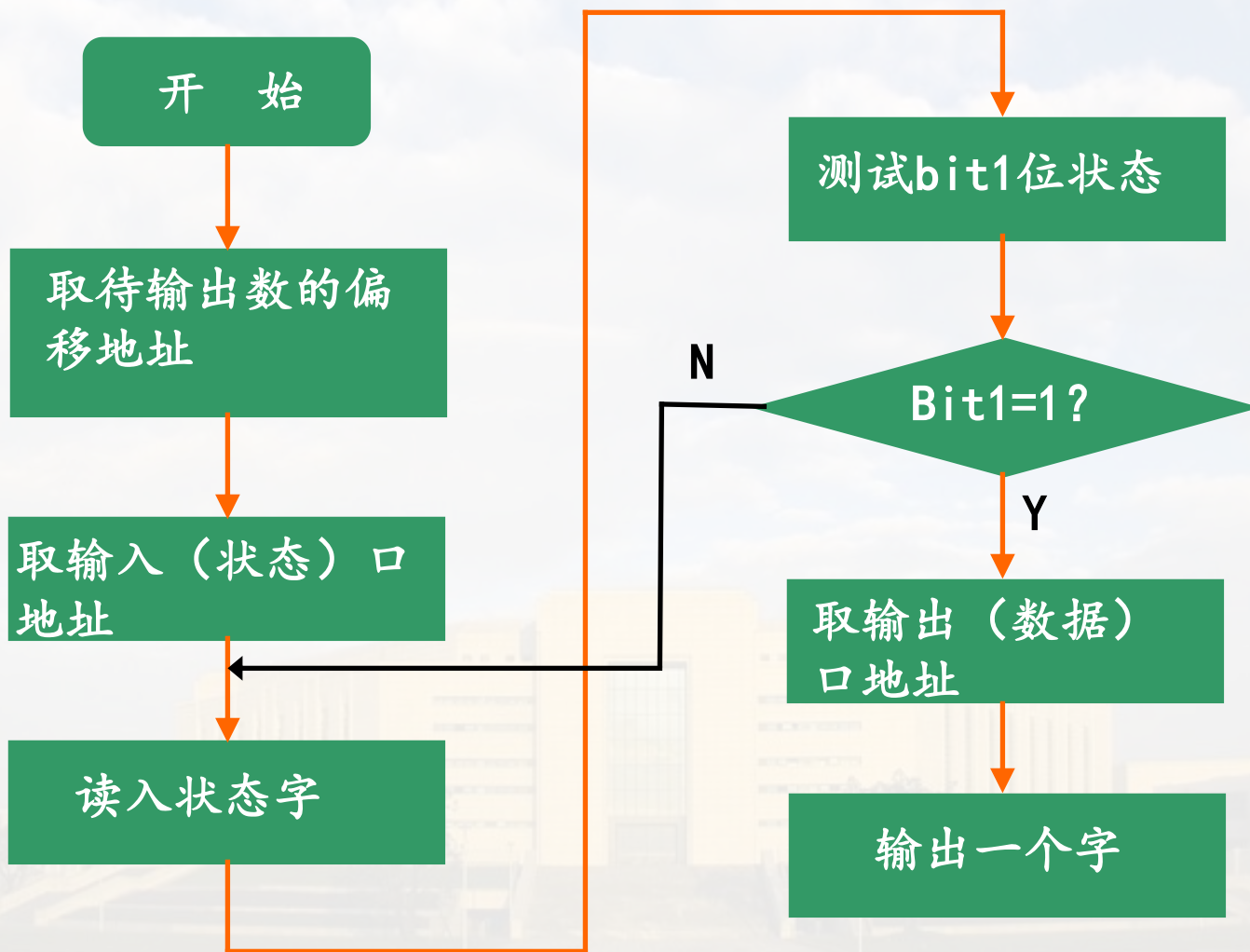
» I/O端口



3.4 逻辑运算和移位指令



“与”指令应用例



3.4 逻辑运算和移位指令

» “与”指令应用例

LEA SI, DATA

MOV DX, 3F8H

超过8位, 间接寻址

WAIT: IN AL, DX

1B的8b : 0b~7b

AND AL, 02H ; 02H=00000010B

JNZ WAIT ; ZF=1 转移

MOV AX, [SI]

MOV DX, 500FH

超过8位, 间接寻址

OUT DX, AX

3.4 逻辑运算和移位指令

» “或”运算指令



格式

OR OPRD1, OPRD2



操作

两操作数按位相“或”，结果送目标地址

3.4 逻辑运算和移位指令



“或”指令的应用

应用

01

实现两操作数 相“或”的运算

- `OR AX, [DI]`

02

使某些位不变，某些位置“1”

- `OR CL, 0FH`

03

在不改变操作数的情况下使 $OF=CF=0$

- `OR AX, AX`

3.4 逻辑运算和移位指令

» “或”指令的应用

OR AL, AL

JPE GOON

否则，最高位置1

OR AL, 80H

GOON: ...

PF=1 转移

3.4 逻辑运算和移位指令

» “或”指令的应用

将一个二进制数9变为
字符 ‘9’



如何实现？

9: 1001B $\longrightarrow$ 39H: 0011, 1001B

» “或”指令的应用

XOR AL, AL ; AL清零

MOV AL, 09H

OR AL, 30H

3.4 逻辑运算和移位指令

» “非”运算指令



格式

NOT OPRD



操作

操作数按位取反再送回原地址

指令中的操作数不能是立即数
指令的执行对标志位无影响



例：

NOT BYTE PTR[BX]

3.4 逻辑运算和移位指令

» “异或”运算指令



格式

`XOR OPRD1, OPRD2`



操作

两操作数按位相“异或”，结果送目标地址（与“1”异或，该位变反）



例：

`XOR BL, 80H` ; 将BL的最高位变反

`XOR AX, AX`

`MOV AX, 0`
; 区别？指令字长不一样

» “测试”指令



格式

TEST OPRD1, OPRD2



操作

执行“与”运算，运算的结果影响标志位，但不送回目标地址。



应用

常用于测试某些位的状态

» 例:

从地址为3F8H的端口中读入一个字节数，当该数的 bit1, bit3, bit5位同时为1时，可从500FH端口将DATA为首地址的一个字输出，否则就不能进行数据传送。编写实现该功能的程序段。

3.4 逻辑运算和移位指令

» 源程序代码

```
LEA SI, DATA
MOV DX, 3F8H
WAIT: IN AL, DX      ; 获取状态端口的信息

TEST AL, 02H      ; 串行测试
JNZ WAIT          ; ZF=1转移
TEST AL, 08H
JNZ WAIT
TEST AL, 20H
JNZ WAIT
MOV AX, [SI]
MOV DX, 500FH
OUT DX, AX        ; 通过数据端口交换信息
```

还有其它测试相应的状态位方式？

3.4 逻辑运算和移位指令

» 源程序代码：三种测试方式

```
LEA SI, DATA
MOV DX, 3F8H
WAIT: IN AL, DX
```

```
TEST AL, 02H ;
JNZ WAIT
TEST AL, 08H
JNZ WAIT
TEST AL, 20H
JNZ WAIT (或? ? ?)
```

串行测试

```
XOR AL, 2AH
JNZ WAIT
```

并行测试一

```
CMP AL, 2AH ;
JNZ WAIT
```

并行测试二

```
MOV AX, [SI]
MOV DX, 500FH
OUT DX, AX
```

逻辑运算指令的比较

AND、OR、XOR指令的特性：

AND: 某位置“0”，某位不变（置“1”）

OR: 某位置“1”，某位不变（置“0”）

XOR: 某位变反（与“1”异或）

» 移位指令



注：

- 移动次数一般用CL来指定；
- 若只移位1次，也可以在指令中直接写常数“1”。

非操作数

3.4 逻辑运算和移位指令



» 非循环移位指令



3.4 逻辑运算和移位指令

» 算术左移和逻辑左移



算术左移指令

SAL OPRD, 1

SAL OPRD, CL



有符号数



逻辑左移指令

SHL OPRD, 1

SHL OPRD, CL

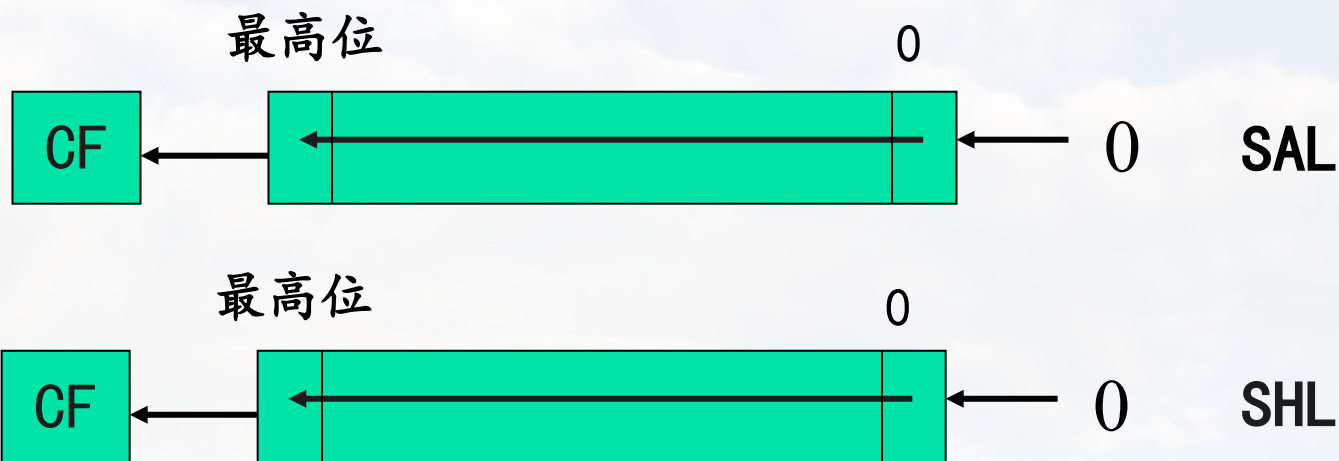


无符号数

注意：在8086/8088
中，实际只有SHL，
无SAL

3.4 逻辑运算和移位指令

» 算术左移和逻辑左移



大家发现这两条指令有什么差别？

相同！

其实在指令系统中它们是同一条指令，正负数
算术左移与逻辑左移结果一样

3.4 逻辑运算和移位指令

» 算术右移和逻辑右移



► 算术右移指令

● SAR OPRD, 1

● SAR OPRD, CL



有符号数



► 逻辑右移指令

● SHR OPRD, 1

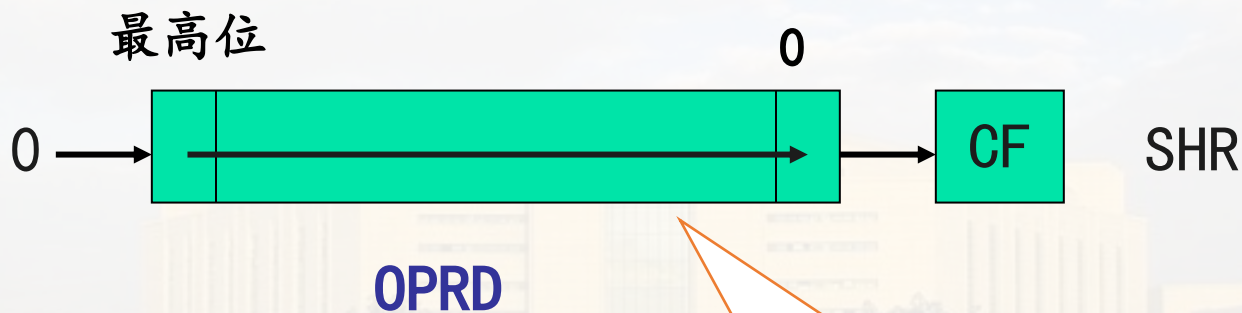
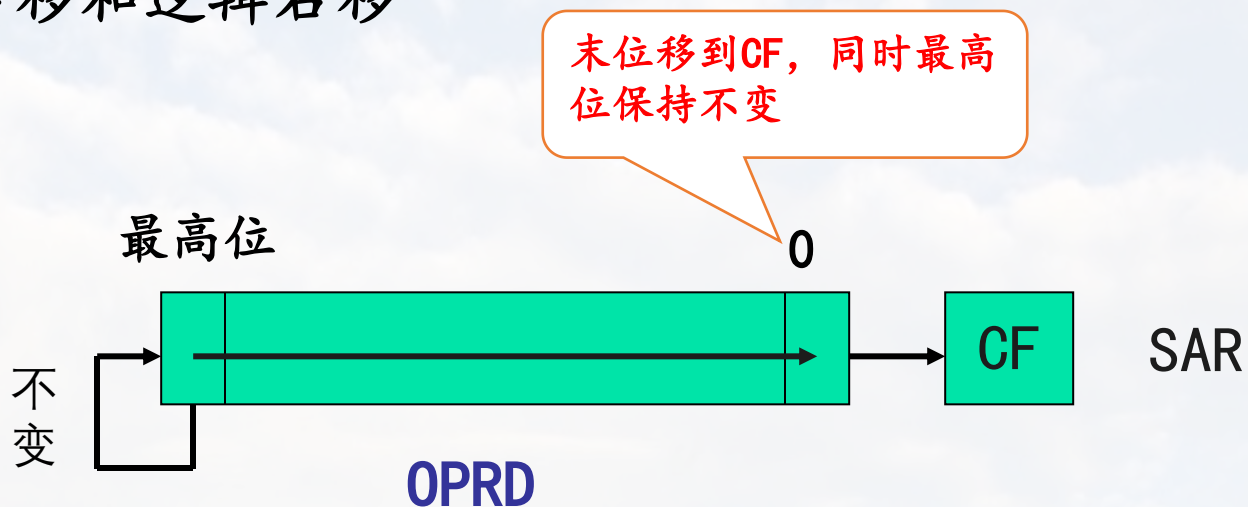
● SHR OPRD, CL



无符号数

3.4 逻辑运算和移位指令

算术右移和逻辑右移



末位移到CF，同时
最高位填0

3.4 逻辑运算和移位指令

» (正数) 算术右移例

- MOV AL, 6AH
- MOV CL, 2
- SAR AL, CL ; 结果为1AH



3.4 逻辑运算和移位指令

» (正数，实际是无符号数首位为0) 逻辑右移例

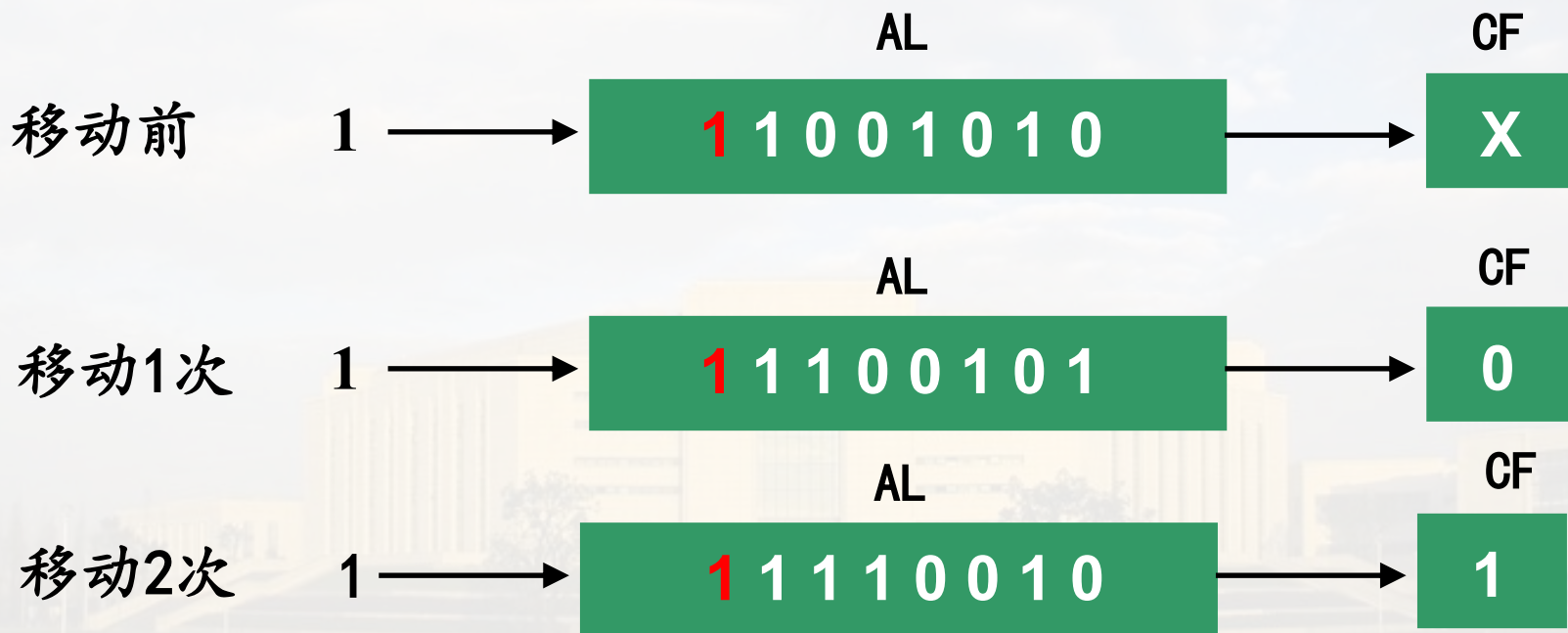
- MOV AL, 6AH
- MOV CL, 2
- SHR AL, CL ; 结果为1AH



3.4 逻辑运算和移位指令

» (负数) 算术右移例

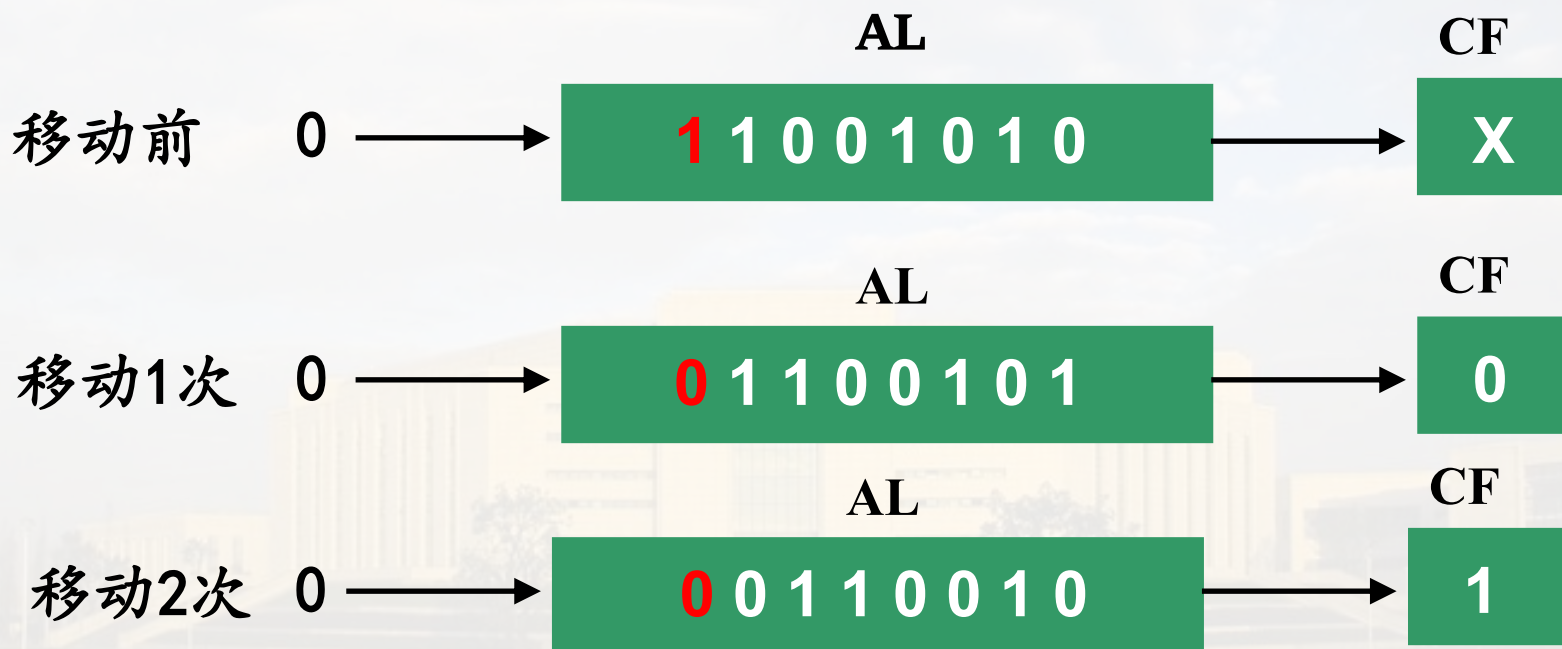
- MOV AL, 0CAH
- MOV CL, 2
- SAR AL, CL ; 结果为0F2H



3.4 逻辑运算和移位指令

» (负数，实际是无符号数首位为1) 逻辑右移例

- MOV AL, 0CAH
- MOV CL, 2
- SHR AL, CL ; 结果为32H



3.4 逻辑运算和移位指令

算术右移和逻辑右移



算术右移指令

SAR OPRD, 1

SAR OPRD, CL



有符号数

首位为0的数，算术右移与逻辑右移结果一样



逻辑右移指令

SHR OPRD, 1

SHR OPRD, CL



无符号数

首位为1的数，算术右移与逻辑右移结果不一样

3.4 逻辑运算和移位指令

非循环移位指令的应用

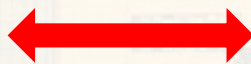
- 左移可实现乘法运算, 右移可实现除法运算
- 指令SAL和SAR, 当移位次为n时, 其作用分别相当于乘以 2^n 和除以 2^n

$$5X \div 2 = (4X + X) \div 2$$

例5 设AX中存放一个无符号数, 若要实现 $(AX) \times 5 \div 2$, 可由以下几条指令完成。

```
MOV    DX, AX
SAL    AX, 1    ; 2X
SAL    AX, 1    ; 4X
ADD    AX, DX   ; 5X
SAR    AX, 1    ; 5X ÷ 2
```

```
MOV    DX, AX
MOV    CL, 2
SAL    DX, CL
ADD    AX, DX
SAR    AX, 1
```



3.4 逻辑运算和移位指令

» 循环移位指令



不带进位位的循环移位(小循环)

CF位

● 左移 ROL

● 右移 ROR



带进位位的循环移位(大循环)

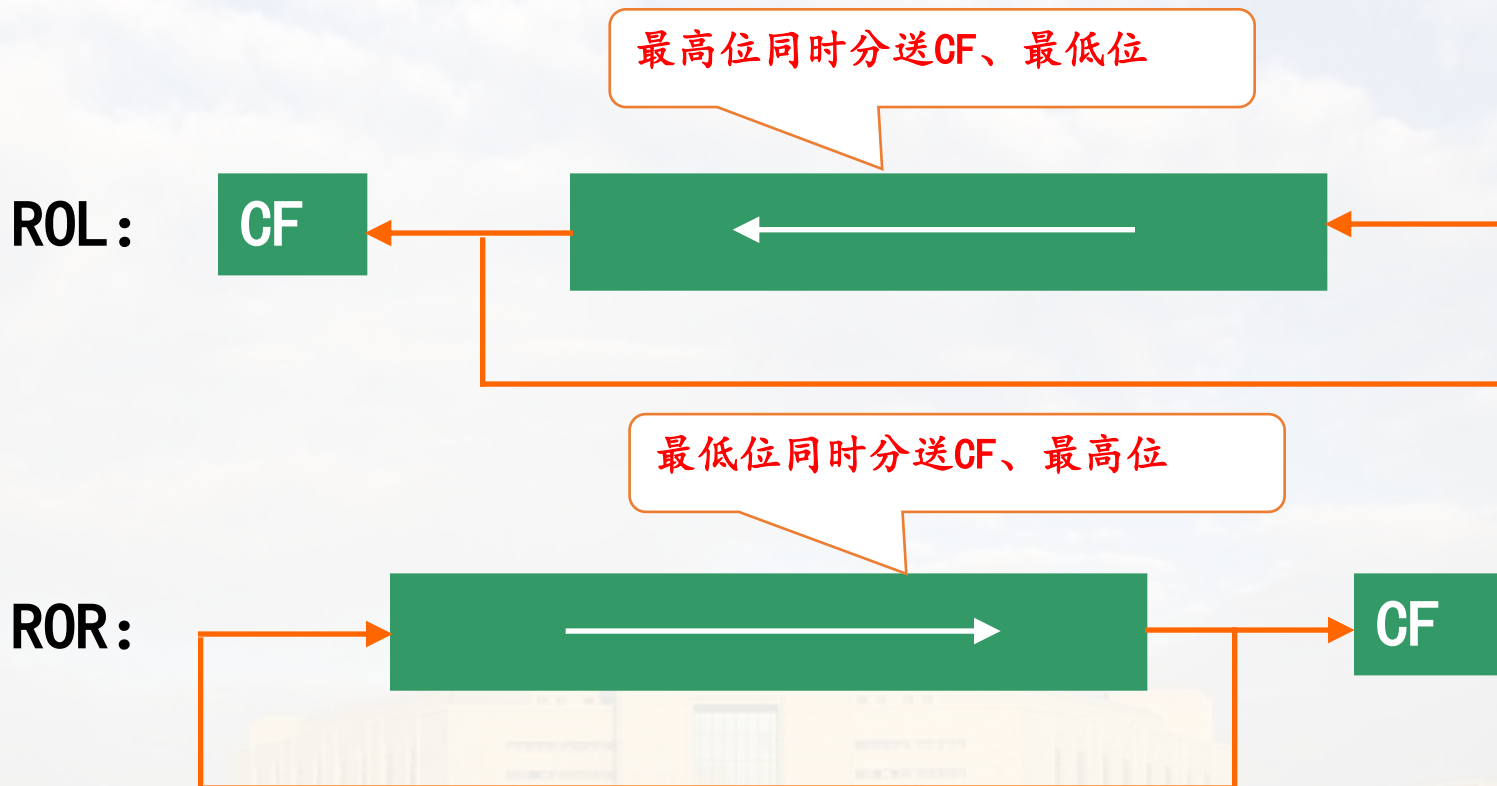
● 左移 RCL

● 右移 RCR

指令格式、对操作数的要求与非循环移位指令相同

3.4 逻辑运算和移位指令

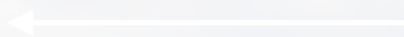
不帶进位位的循环移位（小循环）



3.4 逻辑运算和移位指令

➤ 不带进位位的循环移位（小循环）

格式：ROL/ROR DEST , COUNT



功能：每左移动一位，将移出的最高位送入最低位和CF位；
每右移动一位，将移出的最低位送入最高位和CF位。

3.4 逻辑运算和移位指令



带进位位的循环移位（大循环）





3.4 逻辑运算和移位指令

➤ 带进位位的循环移位（大循环）

格式：RCL/RCR DEST , COUNT

功能：每左移动一位，将CF位送入最低位，同时将移出的最高位送入CF位；

每右移动一位，将CF位送入最高位，同时将移出的最低位送入CF位。

» 循环移位指令的应用

01

用于对某些位状态的测试； P130例3-34

02

高位部分和低位部分的交换；

03

与非循环移位指令一起组成32位或更长字长数的移位。 P130例3-35

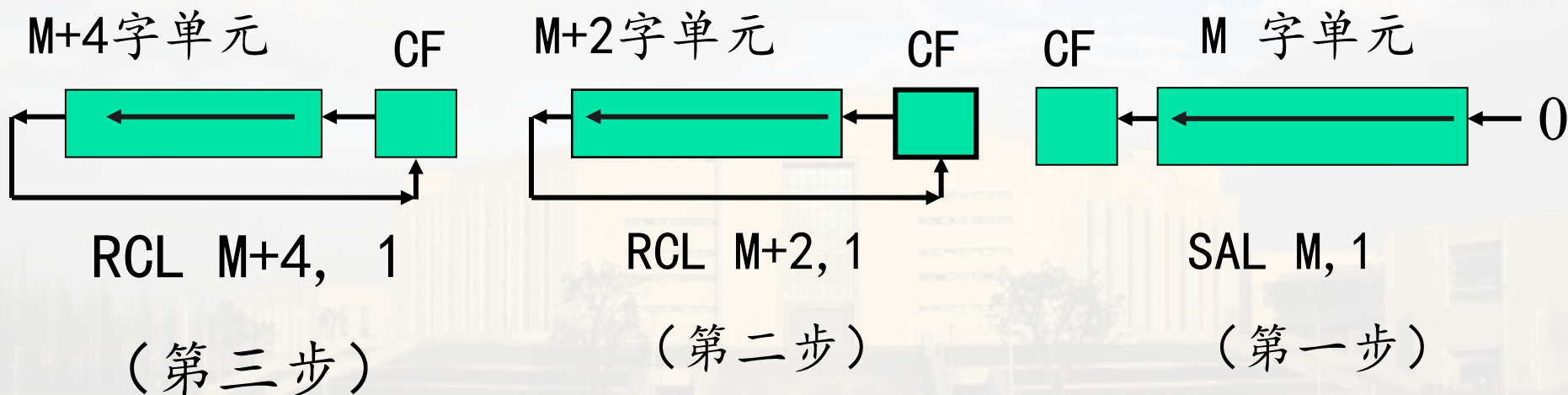
应用

3.4 逻辑运算和移位指令

多字节单元数据联合移位例子

例：下面程序段对从存储单元M开始的三字数据执行左移一位。

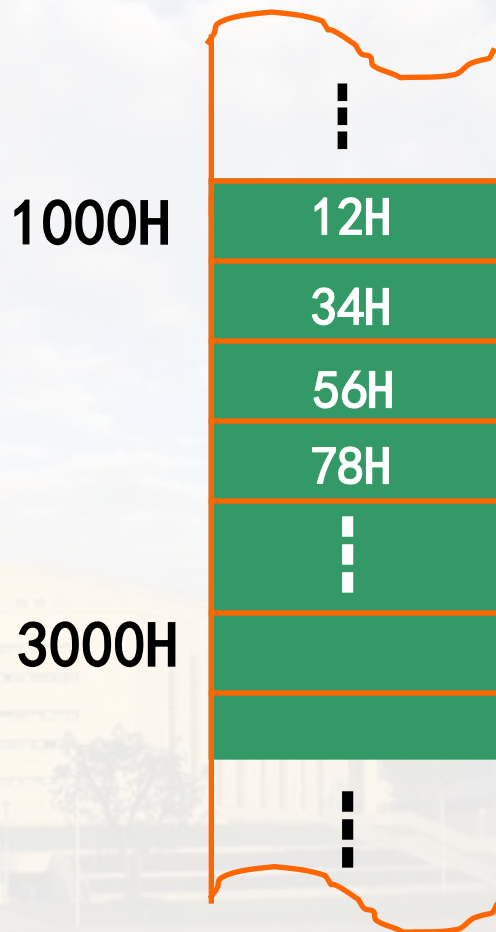
SAL	M, 1	改为:	LEA	SI, M
RCL	M+2, 1		SAL	[SI], 1
RCL	M+4, 1		RCL	[SI+2], 1
			RCL	[SI+2], 1



3.4 逻辑运算和移位指令

» 移位指令应用例子

- 将1000H开始存放的4个压缩BCD码转换为ASCII码存放到3000H开始的单元中去。



3.4 逻辑运算和移位指令

» 移位指令应用例子：



3.4 逻辑运算和移位指令

» 实现程序

MOV SI, 1000H

MOV DI, 3000H

MOV CX, 4

Next: MOV AL, [SI]

AL值备份

MOV BL, AL

AND AL, 0FH

低4位传送

OR AL, 30H

MOV [DI], AL

INC DI

MOV AL, BL

PUSH CX

MOV CL, 4

SHR AL, CL

OR AL, 30H

MOV [DI], AL

INC DI

INC SI

POP CX

DEC CX

JNZ Next

HLT

高4位传送

3.4 逻辑运算和移位指令

» 移位指令应用例子

此例中，CX，CL不能混用；CX指4个BCD码；而CL指4次移位。

12H

56H

78H



» 作业

第三章补充习题：

3.8, 3.9(2), (3), (6), (7)



3.5

串操作指令

串操作指令说明

说明

01

针对数据块或字符串的操作；

02

可实现存储器到存储器的数据传送；

很特殊的用法

03

待操作的数据串称为源串，目标地址称为目标串。

通常情况下，指令不能实现数据在 $M_1 \longleftrightarrow M_2$ 之间传送

3.5 串操作指令

串操作指令的特点

01.

源串地址由DS:[SI]提供，目的串由ES:[DI]提供。

P48

02.

源串允许段重设，目的串不允许段重设。

03.

每次只处理串中的一个单元(字或字节)，并自动修改SI和(或)DI，使其指向下一个单元。

04.

地址修改方向由DF标志位决定：

P145

DF=0 → 增地址方向；

DF=1 → 减地址方向；

05.

数据块长度由CX指定。

06.

指令前面可加上自动重复前缀，以实现自动修改CX内容，实现重复执行操作。



3.5 串操作指令

8086/8088对段存储器使用的约定

序号	内存访问类型	默认段寄存器	可重设的段寄存器	段内偏移地址来源
1	取指令	CS	无	IP
2	堆栈操作	SS	无	SP
3	串操作之源串	DS	ES、SS	SI
4	串操作之目标串	ES	无	DI
5	BP用作基址寻址	SS	ES、SS	按寻址方式计算得有效地址EA
6	一般数据存取	DS	ES、SS	按寻址方式计算得有效地址EA

有效地址EA即《计算机组成原理》课程中的地址

《微机原理》课程中的地址=段地址+段内偏移地址(或EA)

EA: effective address

3.5 串操作指令

重复前缀

不是独立指令，配合串操作指令使用

无条件重复

重复前缀

条件重复

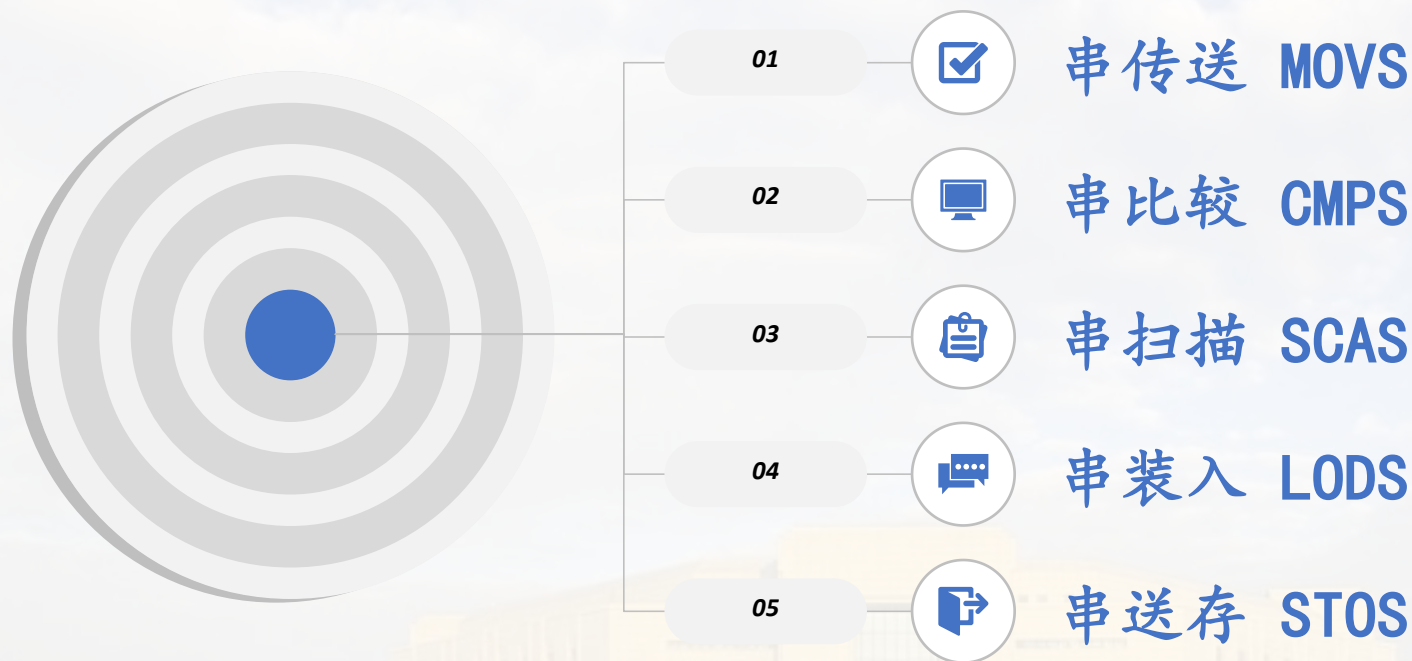
REP → 若 $CX \neq 0$ 则重复

退出条件：执行 CX 次退出；
或 $ZF=1$ ，但 $CX \neq 0$ 退出

退出条件：执行 CX 次退出；
或 $ZF=0$ ，但 $CX \neq 0$ 退出

- | | | |
|---------|-------|------------------------|
| • REPE | 相等重复 | } $CX \neq 0$ 且 $ZF=1$ |
| • REPZ | 为零重复 | |
| • REPNE | 不相等重复 | } $CX \neq 0$ 且 $ZF=0$ |
| • REPNZ | 不为零重复 | |

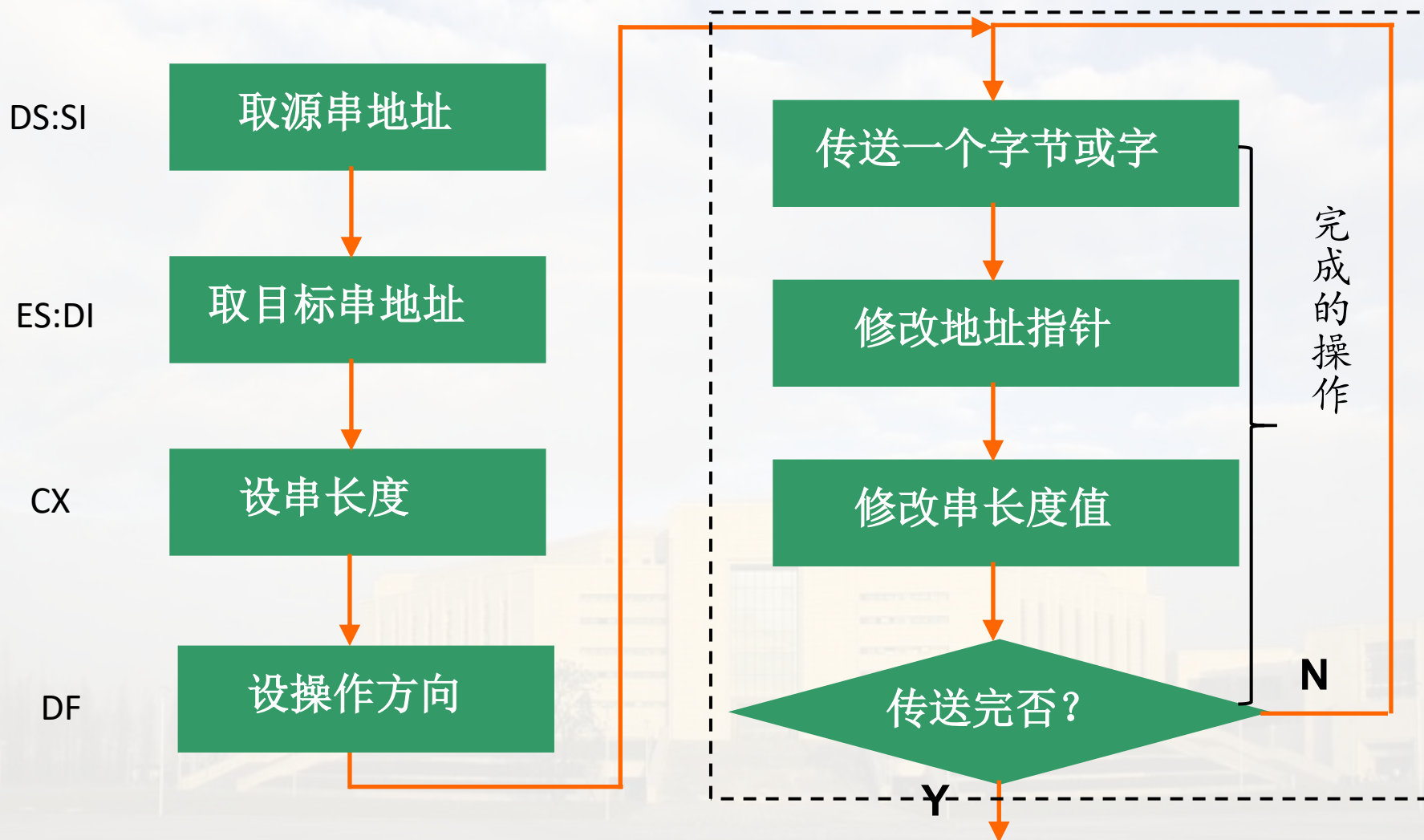
串操作指令



注意：串操作指令中可以出现两个存储器操作数。
这是与其它指令不同的地方。

3.5 串操作指令

串操作指令典型流程(以传送操作为例)



3.5 串操作指令

串传送指令



格式

MOVS
MOVSB
MOVSW

目标串 (ES: DI)

OPRD1, OPRD2

源串 (DS: SI)



说明

- 串传送指令不影响标志位
- 串传送指令常与无条件重复前缀REP连用

能否与条件重复前缀连用？可以，但对标志位无影响，一般不这样使用。

串传送指令的应用例子

对比用MOV指令和MOVS指令实现将200个字节数据从内存的一个区域MEM1送到另一个区域MEM2的程序段。（MEM1、MEM2块之间有间隔，且MEM2地址为高）

P132例3-36，是用MOV指令实现



3.5 串操作指令

串传送指令例

```
LEA SI, MEM1
LEA DI, MEM2
MOV CX, 200
CLD
REP MOVSB
HLT
```

REP等价于:
DEC CX
JNZ NEXT

MOVSB等价于:
MOV AL,[SI]
MOV [DI],AL
INC SI
INC DI

用串传送指令实现200个字节数据的传送:

```
NEXT: MOV AL, [SI]
      MOV [DI], AL
      INC SI
      INC DI
      DEC CX
      JNZ NEXT
      HLT
```

; 参照LEA指令举例

3.5 串操作指令

串比较指令



格式

CMPS

CMPSB

CMPSW

源串 (DS: SI)

目标 (ES: DI)

OPRD1, OPRD2

注意：一般前是目标，后是源，CMPS指令正好相反



说明

- 执行OPRD1-OPRD2, 不送结果。（类似CMP、TEST指令）
- 串比较指令常与条件重复前缀连用，指令的执行不改变操作数，仅影响标志位。
- 重复前缀的操作对标志位不影响

3.5 串操作指令

串比较指令使用例子

比较两组200个字节数据，找出第一个不同数据放入AL，其地址放入BX。

```
LEA SI, MEM1
```

```
LEA DI, MEM2
```

```
MOV CX, 200
```

```
CLD
```

```
REPE CMPSB
```

```
JZ STOP
```

;两组数据完全相同

```
DEC SI
```

；指针已指向下一个单元，应回退，所以减一

```
MOV AL, [SI]
```

```
MOV BX, SI
```

```
STOP: HLT
```


3.5 串操作指令

串比较指令使用例子

比较两组200个字节数据，找出第一个不同数据放入AL，其地址放入BX。（使用其它语句）

```
LEA SI, MEM1
LEA DI, MEM2
MOV CX, 200
CLD
REPE CMPSB
JZ STOP
DEC SI
MOV AL, [SI]
MOV BX, SI
STOP: HLT
```

```
LEA SI, MEM1
LEA DI, MEM2
MOV CX, 200
CLD
MOV AL, [SI]
REPZ CMPS [DI], AL
JZ STOP
DEC SI
MOV AL, [SI]
MOV BX, SI
STOP: HLT
```

可否？

3.5 串操作指令

串扫描指令



格式

SCAS OPRD

SCASB

SCASW

目标串ES: DI

等价于 SCAS ES:[DI], AX/AL



说明

- 执行与AX(或AL)-OPRD, 结果不保存, 只影响标志寄存器。

串扫描指令的应用

说明

常用于在指定存储
区域中寻找某个关
键字。

教材p133例3-38

3.5 串操作指令

串装入 (AL/AX) 指令



格式

LODS OPRD

LODSB

LODSW

源串DS: SI

等价于LODS AL/AX, DS:[SI]



操作

• 对字节: AL ← [DS:SI]

• 对字: AX ← [DS:SI]

» 串装入指令

用于将内存某个区域的数据串依次装入累加器，以便显示或输出到接口。



LODS指令一般不加重复前缀。

教材P134例3-34

3.5 串操作指令

» (AL/AX) 置入串指令类型



格式

STOS OPRD

STOSB

STOSW

目标
操作数

等价于 STOS ES:[DI] , AL/AX



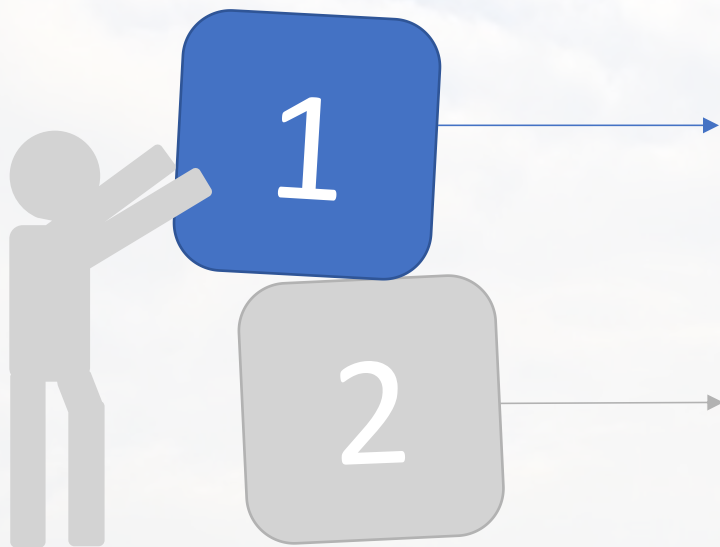
操作

• 对字节: AL → [ES:DI]

• 对 字: AX → [ES:DI]

3.5 串操作指令

» (AL/AX) 置入串指令类型



常用于将内存某个区域置同样的值:如清零, 置同一个值

此时:

- 将待送存的数据放入AL (字节数) 或AX (字数据);
- 确定操作方向 (增地址/减地址) 和区域大小 (串长度值);
- 使用串存储指令+无条件重复前缀, 实现数据传送。

串存储指令例

说明

将内存某单元清零

教材P135例3-40

3.5 串操作指令

串操作指令应用注意事项

01.

需要定义附加段

- 目标操作数必须在附加段

02.

需要设置数据的操作方向

- 确定DF的状态：CLD/STD

03.

源串和目标串指针分别为SI和DI

04.

串长度值必须由CX给出

注意重复前缀的使用方法

05.

- 传送类指令前加无条件重复前缀（MOVS、STOS）
- 串比较类指令前加条件重复前缀，但前缀不影响ZF状态（CMPS、SCAS）



串操作指令应用注意事项

01. MOVSB: 源串 (组) $\xrightarrow{\text{单向传送}}$ 目标串 (组)

02. CMPSB: 源串 (组) $\xleftrightarrow{\text{双向比较}}$ 目标串 (组)

03. SCASB: 单 (AL/AX) $\xrightarrow[\text{检索关键词}]{\text{单向、区域}}$ 目标串 (组)

04. LODSB: 源串 (组) $\xrightarrow{\text{单向、逐个传送}}$ 单 (AL/AX)

05. STOSB: 单 (AL/AX) $\xrightarrow[\text{相同设置}]{\text{单向、区域}}$ 目标串 (组)

3.6

程序控制指令



» 程序的执行方向

说明

01

程序控制类指令的本质：

- 控制程序的**执行方向**

02

决定程序执行方向的因素：

- **CS、IP**，它们决定了从哪里取指令，也即程序执行顺序或方向。

03

控制程序执行方向的方法：

- **修改CS和IP**，则程序转向另一个代码段（另一个CS段）执行；**即段间**
- **仅修改IP**，则程序将改变当前的执行顺序，转向本代码段内其它某处执行；**即段内**

3.6 程序控制指令

» 转移指令



转移指令：修改指令指针IP的值（即段内）或修改段寄存器CS及指令指针IP的值（即段间），实现程序的转移。



- 无条件转移指令

- 无条件转移到目标地址，
执行新的指令

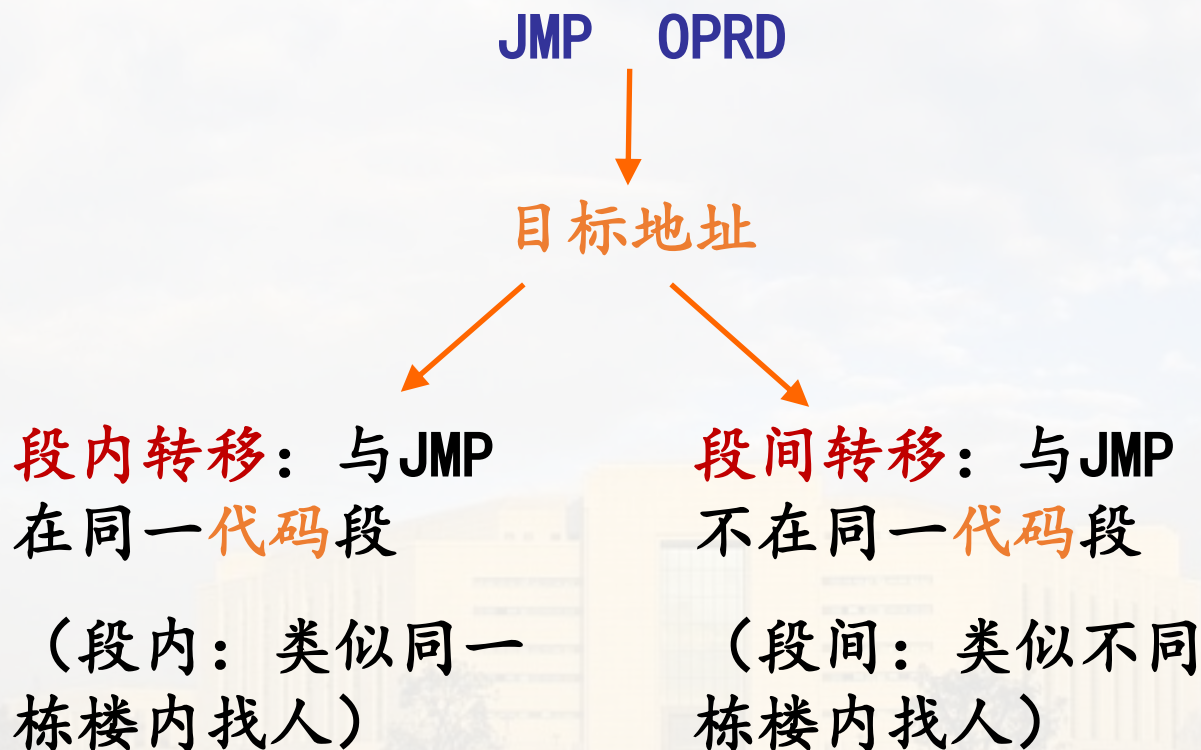
- 有条件转移指令

- 在满足一定条件的情况下才转移到目标地址

3.6 程序控制指令

无条件转移指令

■ 格式:



原则上可实现在整个内存空间的转移

3.6 程序控制指令

无条件段内转移

- 转移的目标地址在当前代码段内，段地址不改变，仅改变IP。
- 即：目标地址是8/16位偏移地址。

指令中直接给出目标地址

段内直接转移

常数、标号，如：1000H, label

由指令中的寄存器或存储器操作数指出目标地址

段内间接转移

16位寄存器或存储器字单元中，
如：BX, WORD PTR[BX]

3.6 程序控制指令

» 段内直接转移



说明

转移的目标地址由指令直接给出, 如: 常数 (8/16位)、标号



格式

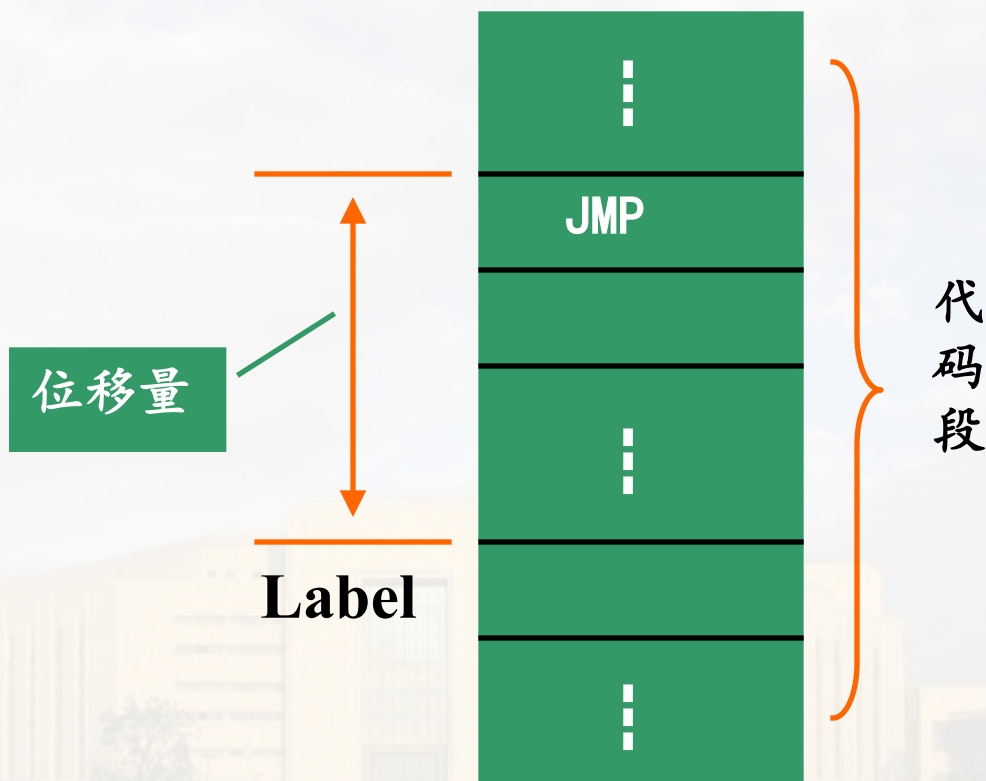
JMP (Near) Label

$IP(新) = IP(旧) + \text{相对位移量 (8/16位)}$

近地址标号

» 段内直接转移示意图

JMP Label



下一条要执行指令的偏移地址，即新IP=当前IP+位移量

3.6 程序控制指令

» 段内间接转移

■ 段内间接转移

- 转移的目标地址存放在某个**16位**通用寄存器或存储器的**某字单元（即16位）**中。

格式：

JMP (Near) OPRD

- OPRD (**16位**通用寄存器内)：**IP(新)=16位通用寄存器的内容**
- OPRD (存储器的两字节单元内)：**IP(新)=存储器两字节单元内容**

3.6 程序控制指令

» 段内间接转移

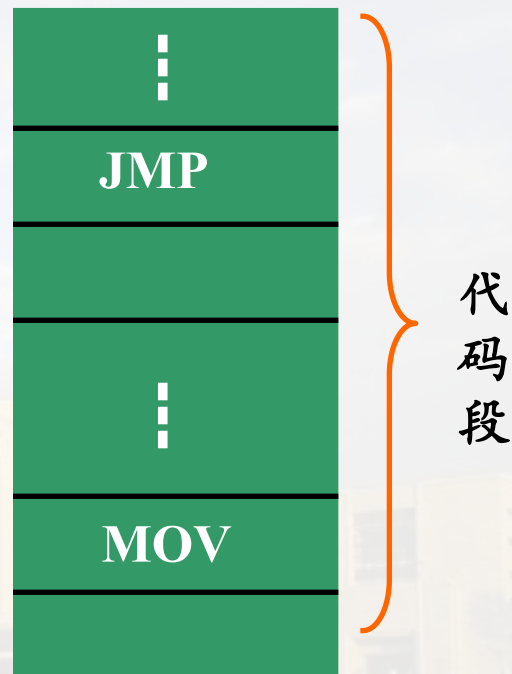
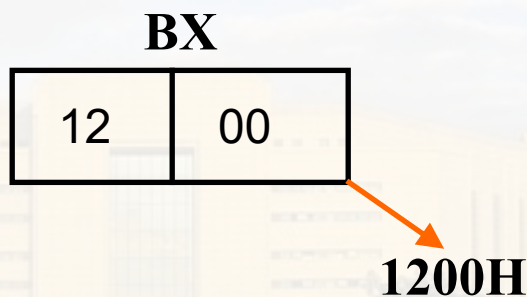
■ 例1：（转移地址：16位通用寄存器内）

■ **JMP BX**

■ 若：BX=1200H

■ 则：转移的目标地址=1200H

覆盖原IP，即现IP=1200H



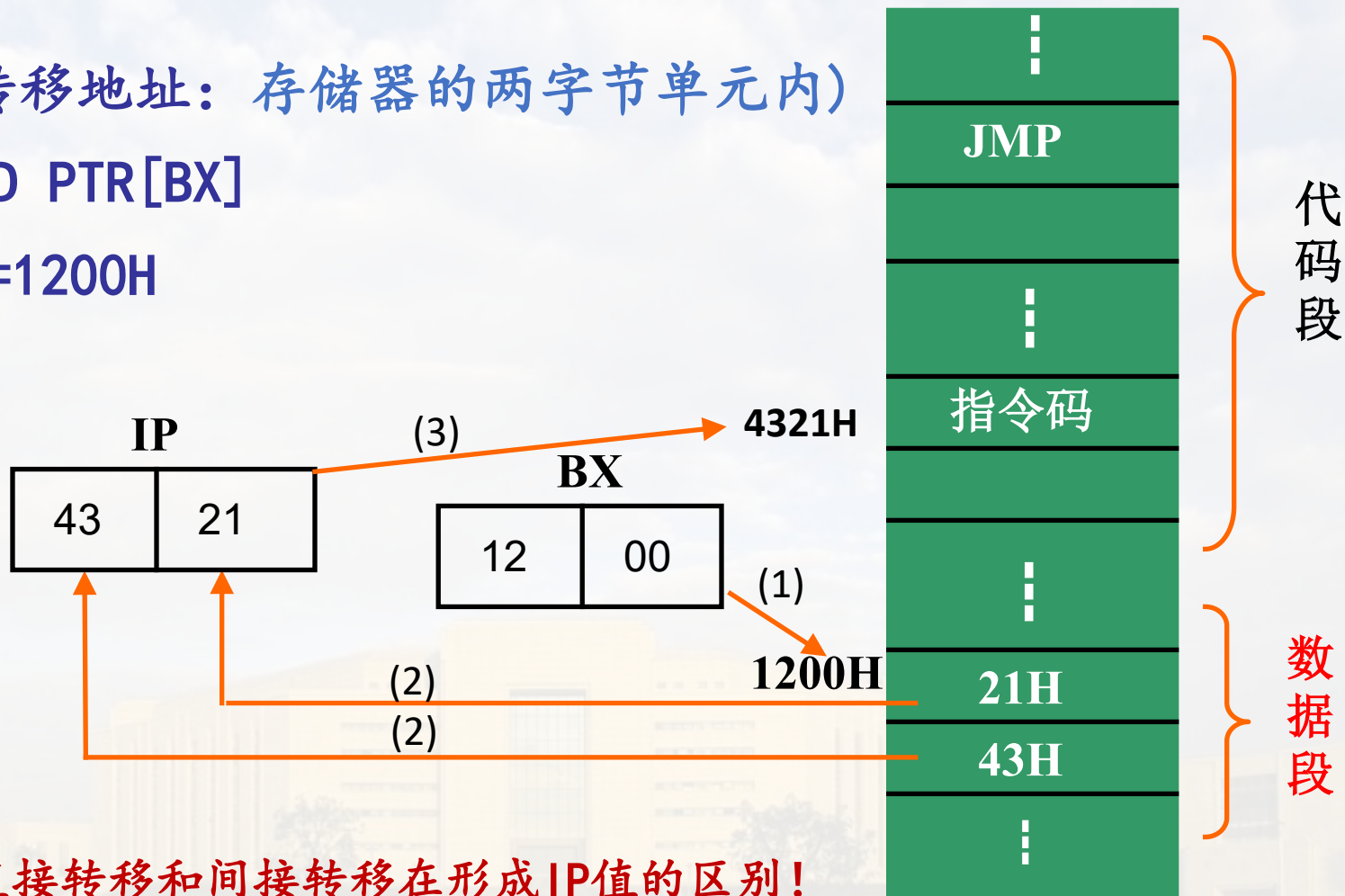
3.6 程序控制指令

段内间接转移

■ 例2: (转移地址: 存储器的两字节单元内)

■ `JMP WORD PTR[BX]`

设: `BX=1200H`



注意: 段内直接转移和间接转移在形成IP值的区别!

3.6 程序控制指令

» 段内间接转移

- 例3：（转移地址：存储器的两字节单元内）
 - `JMP WORD PTR[BX+DI]`
 - 若： $DS=3000H$, $BX=1300H$, $DI=1200H$, $[32500H]=50H$,
 $[32501H]=23H$
 - 则转移的目标地址 $IP(新)=2350H$
 - （参看P136）

无条件段间转移

- 转移的目标地址不在当前代码段内。
- 目标地址为32位，包括段地址和偏移地址。

指令中直接给出目标地址

段间直接转移

由指令中的32位（双字）存储器单元给出目标地址

段间间接转移

能否由R给出？为何？

» 段间直接转移



说明

转移的目标地址由指令直接给出



格式

JMP **FAR** Label

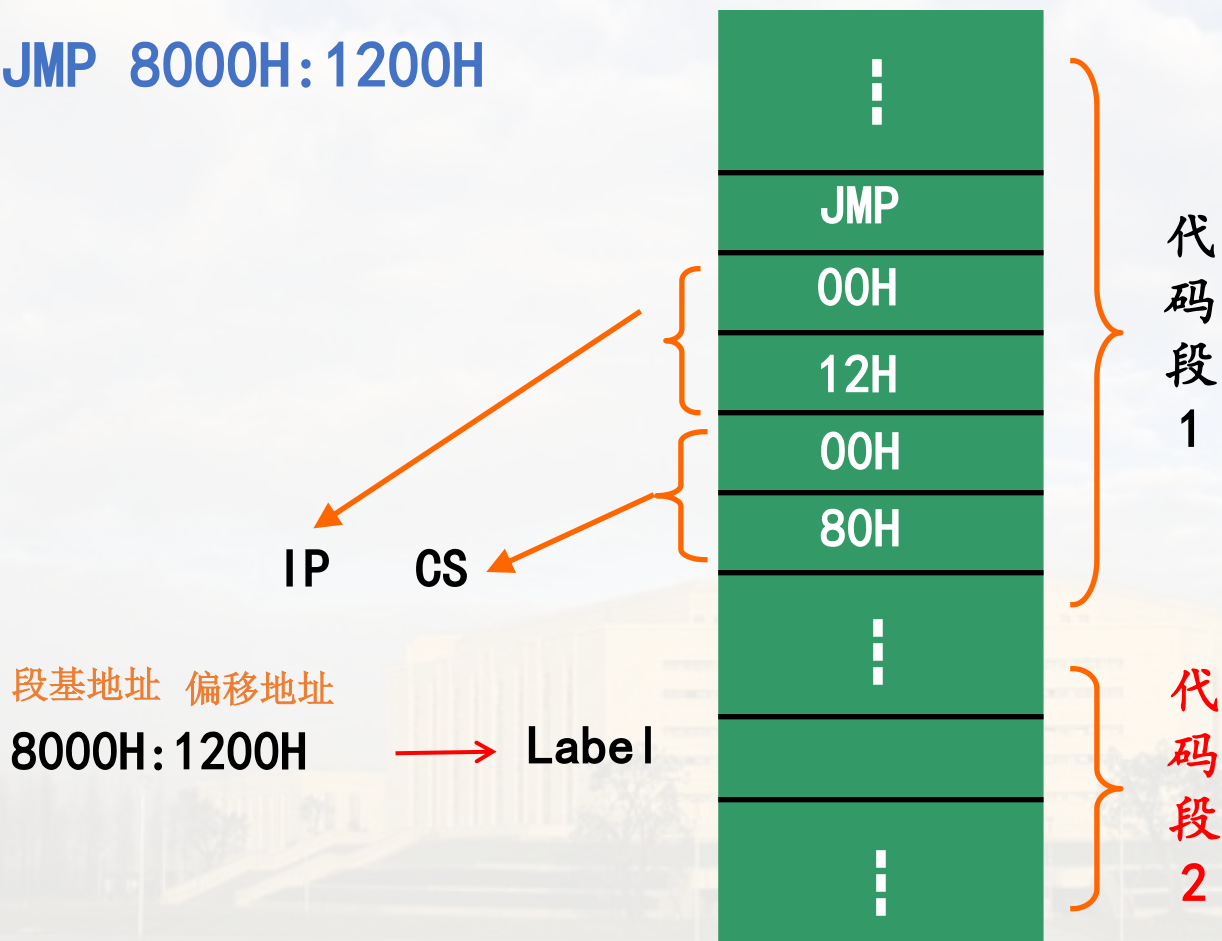
远地址标号

3.6 程序控制指令

段间直接转移示意图

例：

JMP 8000H:1200H

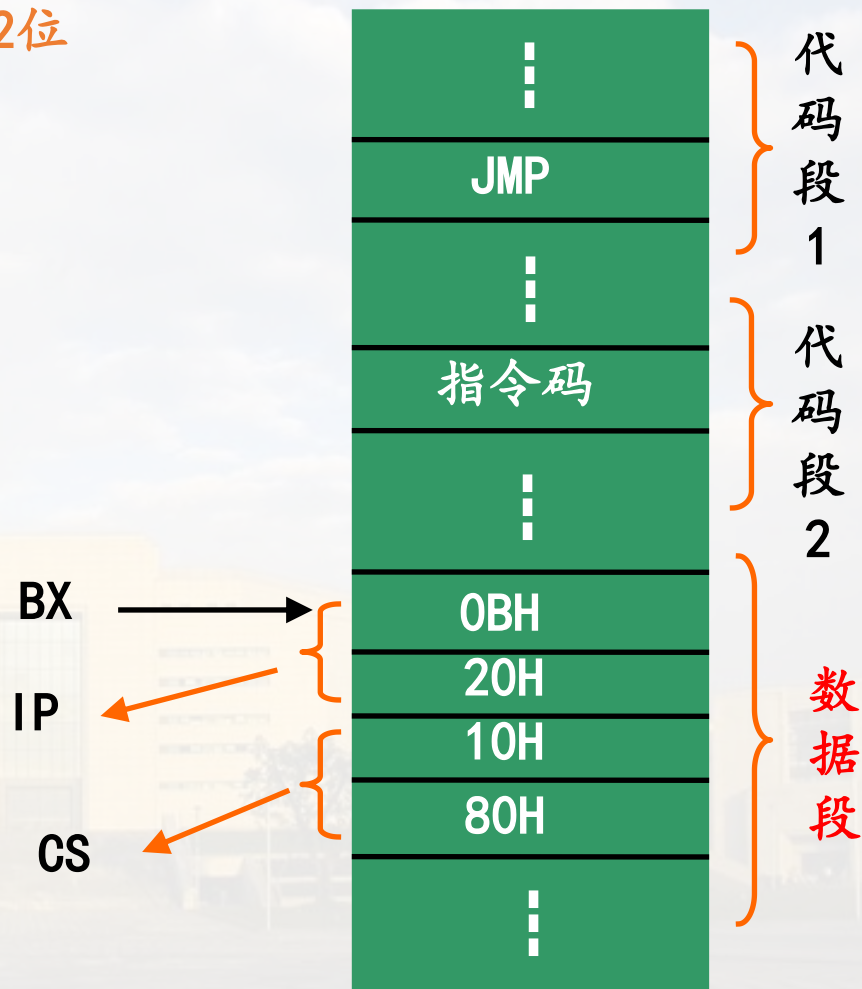


3.6 程序控制指令

段间间接转移

■ 段间间接寻址

- 转移的目标地址由指令中的**32位存储器单元**给出
- 例：
- `JMP DWORD PTR[BX]`
- （参看P137例3-42）



3.6 程序控制指令

无条件转移指令例

(1) 2000:0100

MOV AX, 1200H

(2) 2000:0103

JMP NEXT

(3) 2000:0120

NEXT: MOV BX, 1200H

(4)

JMP DWORD PTR [BX]

(5) 2000:1200

两条JMP指令分别采用的是哪种寻址？

无条件/有条件

段内/段间

直接/间接转移

无条件转移指令例

(1) 2000:0100

MOV AX, 1200H

(2) 2000:0103

JMP NEXT

无条件段内直接转移指令

(3) 2000:0120

NEXT: MOV BX, 1200H

(4)

JMP DWORD PTR [BX]

无条件段间间接转移指令

(5) 2000:1200

两条JMP指令分别采用的是哪种寻址？

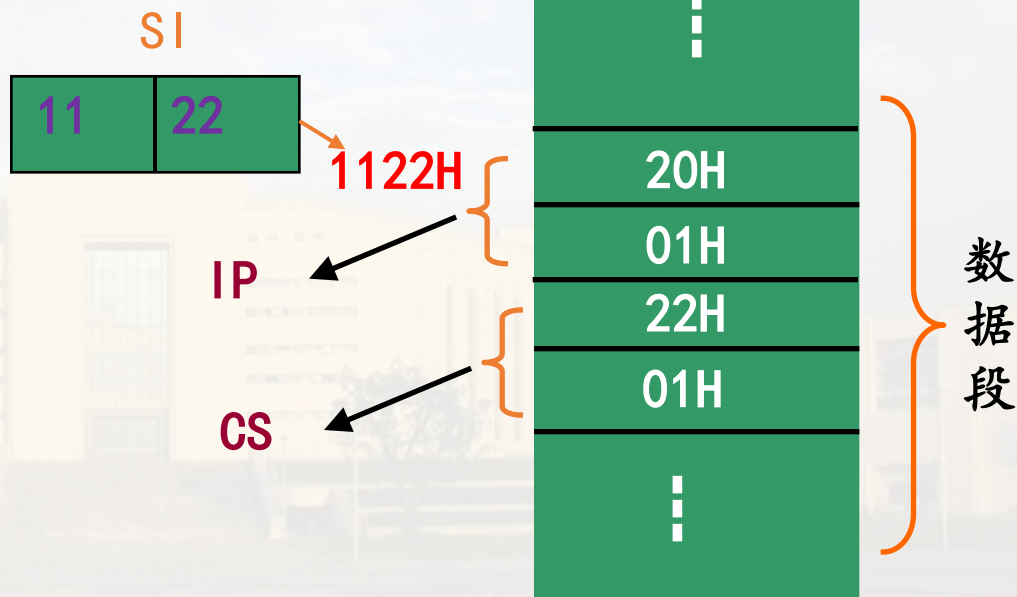
3.6 程序控制指令

无条件段间间接转移指令例

- 例：将DS段内偏移地址为1122H开始的地址内容作为无条件段间间接转移地址的起始地址，为其赋初值如下CS: IP=0122H:0120H。

- `MOV SI, 1122H`
- `MOV WORD PTR[SI], 0120H`
- `ADD SI, 2`
- `MOV WORD PTR[SI], 0122H`
- `JMP WORD PTR[SI]`

程序是否正确？



3.6 程序控制指令

无条件段间间接转移指令例

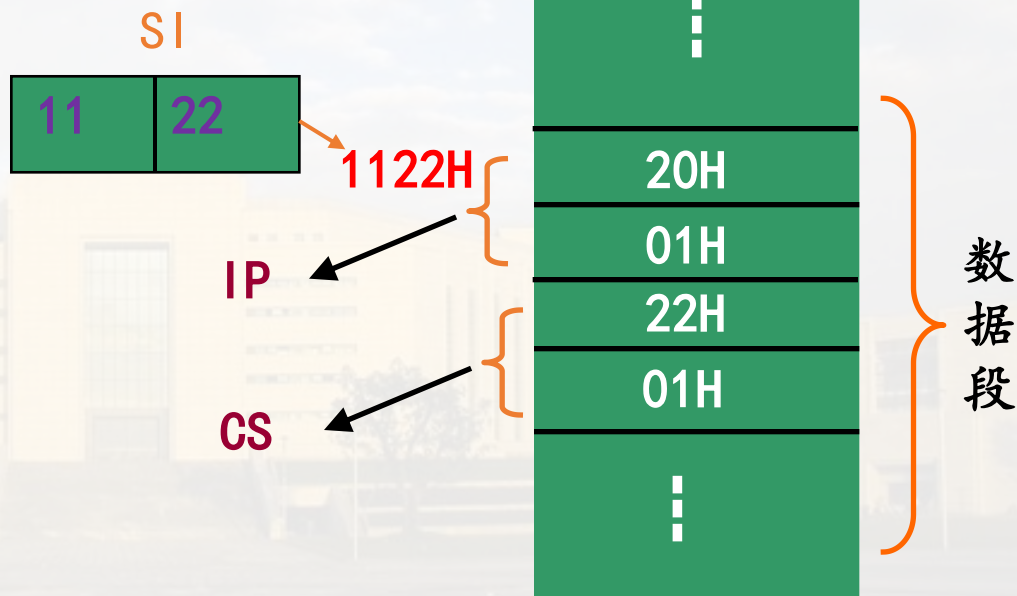
- 例：将DS段内偏移地址为1122H开始的地址内容作为无条件段间间接转移地址的起始地址, 为其赋初值如下CS: IP=0122H:0120H。

- `MOV SI, 1122H`
- `MOV WORD PTR[SI], 0120H`
- `ADD SI, 2`
- `MOV WORD PTR[SI], 0122H`
- `JMP WORD PTR[SI]`

程序是否正确?

最后一条语句改为:

`JMP DWORD PTR[SI-2]`



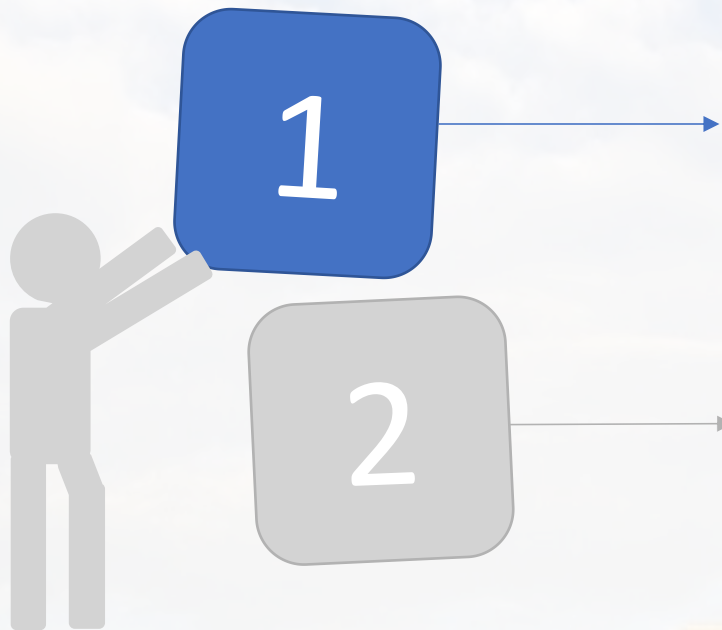
3.6 程序控制指令

➤ 比较以下三类指令：操作数指令、输入输出指令、转移指令

- | | |
|--------|---|
| 操作数指令 | ■ 1. MOV AX , <u>[1000H]</u> ;直接寻址 |
| | ■ 2. MOV AX , <u>BX</u> ;寄存器直接寻址 |
| | ■ 3. MOV AX , <u>[BX]</u> ;寄存器间接寻址, DS: [BX] |
| 输入输出指令 | ■ 4. 1) IN AX/AL, <u>PORT</u> (8位地址) ;直接寻址 |
| | ■ 2) OUT <u>PORT</u> (8位地址), AX/AL ;直接寻址 |
| | ■ 5. 1) MOV <u>DX</u> , <u>PORT</u> (16位地址) |
| | ■ IN AX/AL, <u>DX</u> ;间接寻址 |
| | ■ 2) MOV <u>DX</u> , <u>PORT</u> (16位地址) |
| | ■ OUT <u>DX</u> , AX/AL ;间接寻址 |
| 转移指令 | ■ 6. JMP <u>1000H</u> ;段内直接转移, 指令执行后, IP (新) = IP (旧) + 1000H |
| | ■ 7. JMP <u>BX</u> ;段内间接转移, 指令执行后, IP=BX |
| | ■ 8. JMP <u>WORD PTR []</u> ;段内间接转移, 指令执行后, IP=存储器两字节单元内容 |
| | ■ 9. JMP <u>DWORD PTR []</u> ;段间间接转移, 指令执行后, IP=存储器低两字节单元内容, CS=存储器高两字节单元内容 |

3.6 程序控制指令

» 条件转移指令



在满足一定条件下，程序转移到目标地址继续执行

条件转移指令均为**直接寻址**的**段内短转移**，即转移的位移量为8位补码表示，范围为：

$-128 \text{-----} +127$

与无条件直接寻址不同：8/16位

所有的条件转移指令见教材P138表3-4.

3.6 程序控制指令

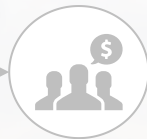
条件转移指令的应用

几种条件转移指令的应用



JC/JNC

- 判断CF的状态。常用于比大小



JZ/JNZ

- 判断ZF的状态。常用于循环体的结束判断



JO/JNO

- 判断OF的状态。常用于有符号数溢出的判断



JP/JNP

- 判断PF的状态。用于判断运算结果低8位中1的个数是否为偶数



JA/JAE/JB/JBE

- 判断CF或CF+ZF的状态。常用于无符号数的大小比较



JG/JGE/JL/JLE

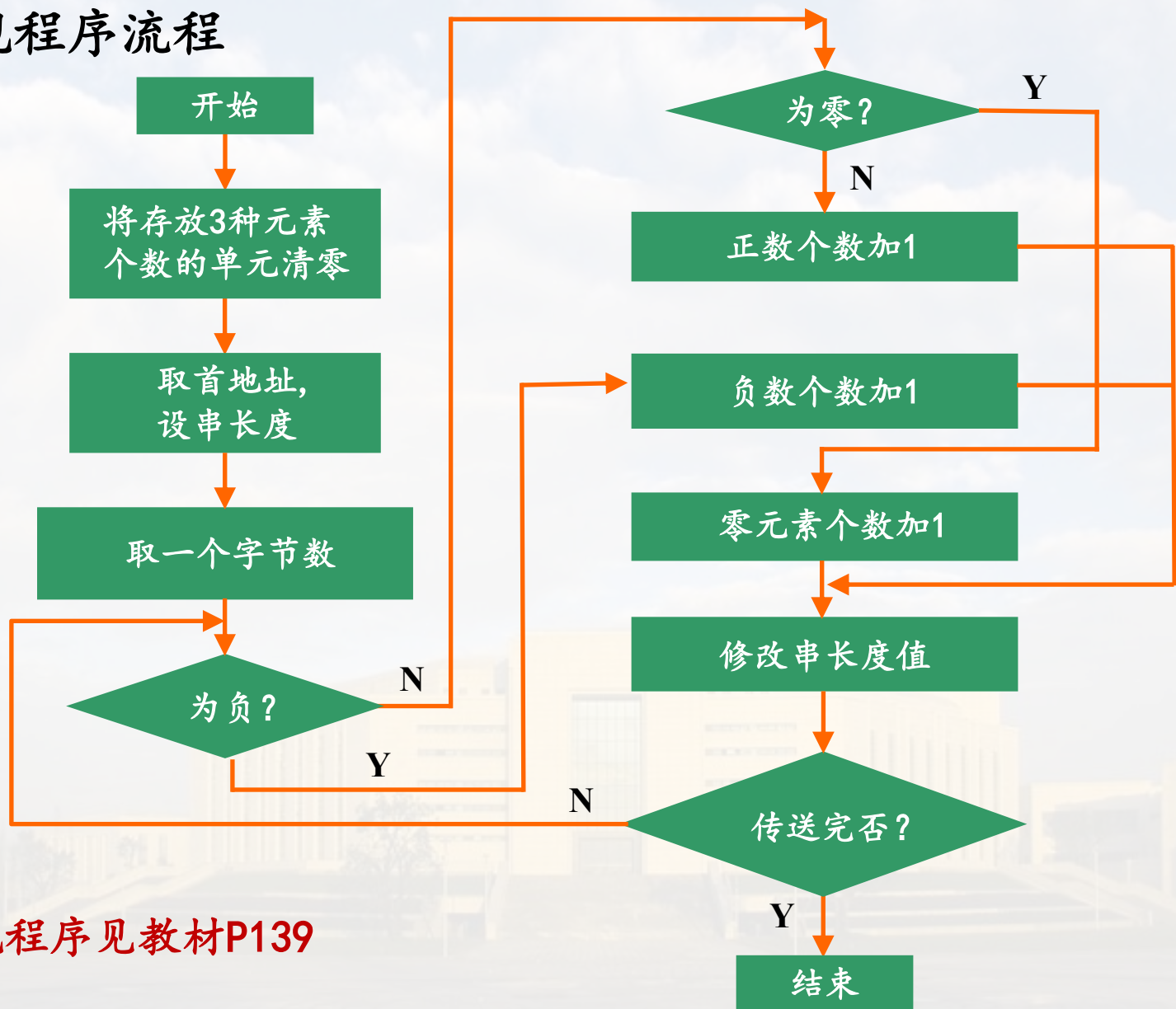
- 判断SF、OF和ZF的状态。常用于带符号数的大小比较

» 转移指令例

统计内存数据段中以TABLE为首地址的100个8位有符号数中正数、负数和零的个数。

3.6 程序控制指令

实现程序流程



实现程序见教材P139

3.6 程序控制指令

转移指令应用总结:

转移指令

无条件转移指令

无条件段内转移指令
(仅改变IP)

无条件段内直接转移指令格式: **JMP** 常数/标号 (8/16位)

无条件段内间接转移指令格式: **JMP** 寄存器/存储器操作数 (16位)

无条件段间转移指令
(既改变IP, 又改变CS, 32位)

无条件段间直接转移指令格式: **JMP FAR** 段基地址: 段内偏移地址

无条件段间间接转移指令格式: **JMP FAR** **DWORD** PTR[], 即32位 (双字) 存储器单元

有条件转移指令: 直接寻址, 段内短转移, 即8位

3.6 程序控制指令

» 循环控制指令

循环范围：

- 以当前IP为中心的-128~+127范围内循环。

只能段内转移，与PPT92的条件转移指令一样

循环次数由CX寄存器指定。

说明

循环指令：

LOOP → 无条件循环指令

{ *LOOPZ
*LOOPNZ } 条件循环指令

判断条件：CX ≠ 0

判断条件：ZF=1，或ZF≠1，均且CX≠0

3.6 程序控制指令

» 无条件循环指令



格式

LOOP LABEL



循环条件

CX $\neq$ 0



操作：等价于

- DEC CX ; CX-1 $\rightarrow$ CX
- JNZ LABEL (符号地址)

; CX $\neq$ 0则转LABEL，否则执行下条指令

3.6 程序控制指令

带条件循环指令



格式

LOOPZ LABEL 或 LOOPE LABEL



循环条件: $CX \neq 0$, 且 $ZF=1$

退出循环: $CX = 0$, 或 $ZF=0$



操作: 等价于

- $DEC\ CX ; CX-1 \rightarrow CX$
- $JZ\ LABEL\ (\text{符号地址}) ; CX \neq 0, \text{且} ZF=1 \text{继续循环, 否则, 则转} LABEL, \text{执行下条指令}$

带条件循环指令



格式

LOOPNZ LABEL 或 LOOPNE LABEL



循环条件: $CX \neq 0$, 且 $ZF=0$

退出循环: $CX=0$, 或 $ZF=1$

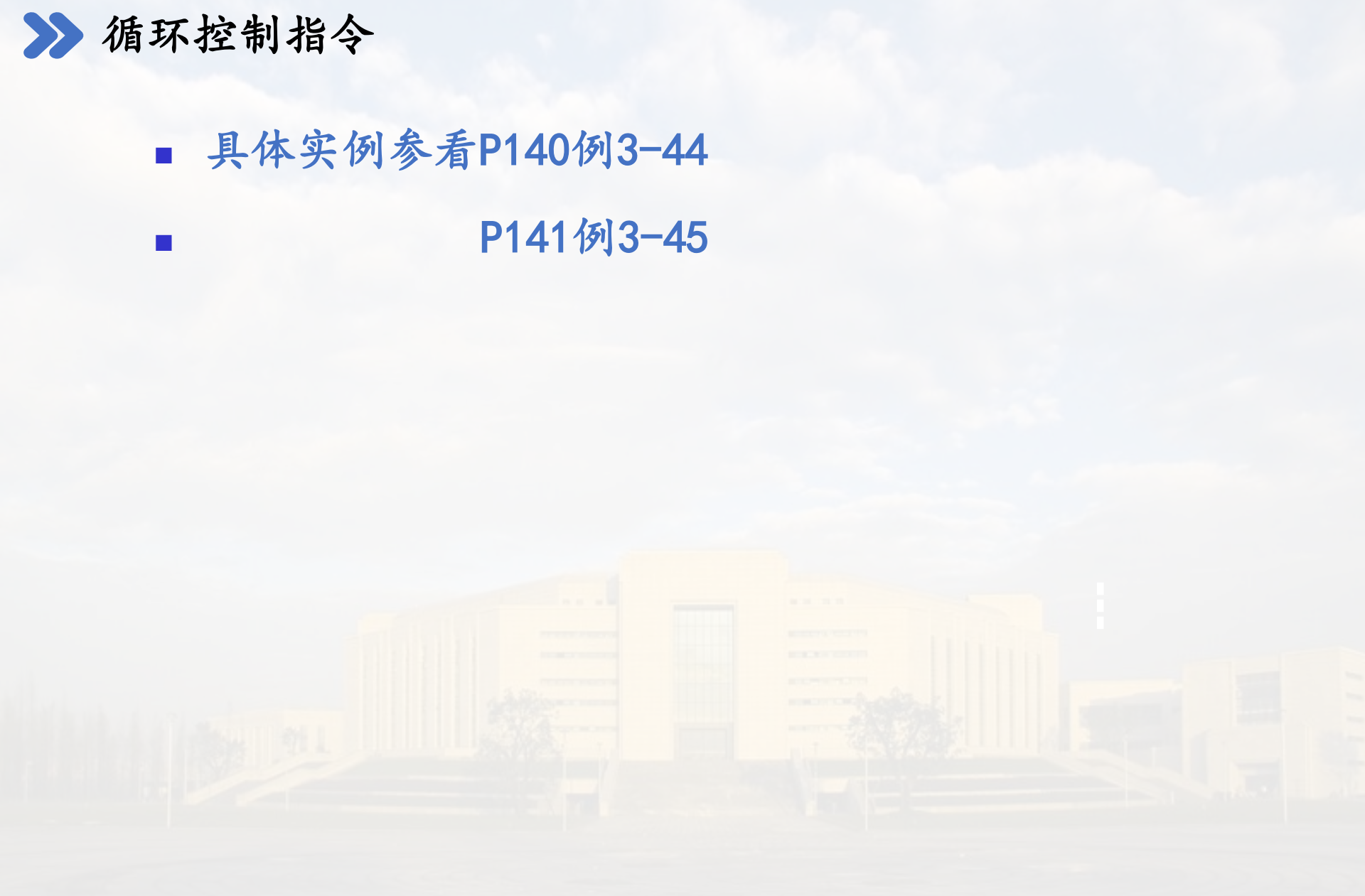


操作: 等价于

- `DEC CX ;  $CX-1 \rightarrow CX$`
- `JNZ LABEL (符号地址) ;  $CX \neq 0$ , 且  $ZF=0$  继续循环, 否则, 则转 LABEL, 执行下条指令`

» 循环控制指令

- 具体实例参看P140例3-44
- P141例3-45

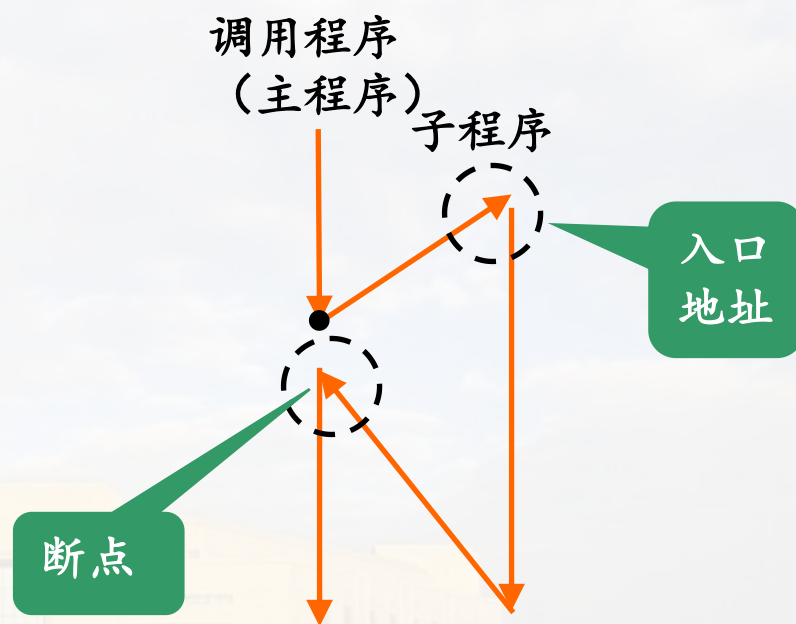


3.6 程序控制指令

过程调用和返回

参考PPT的P74“程序控制指令中的无条件转移指令”

- 用于调用一个子过程；
- 子过程由程序员预先设计并装入内存
- 子过程执行结束后要返回原调用处
- 有两个重要的概念：**入口地址**和**断点**。



调用指令的执行过程



保护断点；

- 将调用指令的下一条指令的地址（断点）压入堆栈

自动压栈，硬件实现，不需编程



获取子过程的入口地址；

- 子过程第1条指令的地址

将新地址送IP或CS+IP中



执行子过程，含相应参数的保存及恢复；



恢复断点，返回原程序。

- 将断点地址（IP或CS+IP）由堆栈弹出

» 过程调用

- 段间直接调用
- 段间间接调用

段间调用

过程调用

段内调用

- 段内直接调用
- 段内间接调用

参考PPT的P76“程序控制指令中的无条件转移指令”

3.6 程序控制指令

段内调用

- 被调用程序与调用程序在**同一代码段**
 - 调用前只需保护断点的偏移地址（即IP, 2个字节）

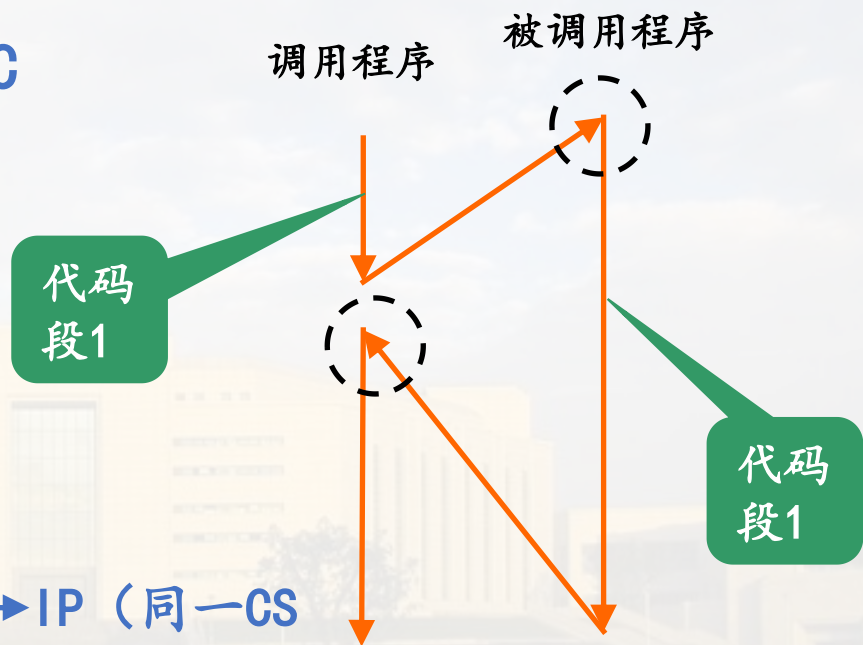
- 格式：

- `CALL NEAR PROC`

近过程名

- 执行过程：

- 将断点的偏移地址压入堆栈
- 根据过程名找子程序入口 → IP（同一CS段）



3.6 程序控制指令

» 段内调用例

(1) CALL TIMER → 直接调用

IP = 当前IP + 16位位移量 (TIMER) 而无条件段内转移 (8 / 16位)

(2) CALL WORD PTR[SI] → 间接调用

16位, 单字

设: SI=1200H
CS=6000H

执行第 (2) 条指令后:

CS = 6000H

段内调用, 段地址不变

IP = 3344H

SI

1200H

12

00

3344H

②

①

CALL

子过程

44H

33H

代码段

数据段

» 段间调用

 子过程与原调用程序不在同一代码段



调用前需保护断点的段基址和偏移地址，4字节

 先将断点的CS压栈，再压入IP。

3.6 程序控制指令

» 段间调用例

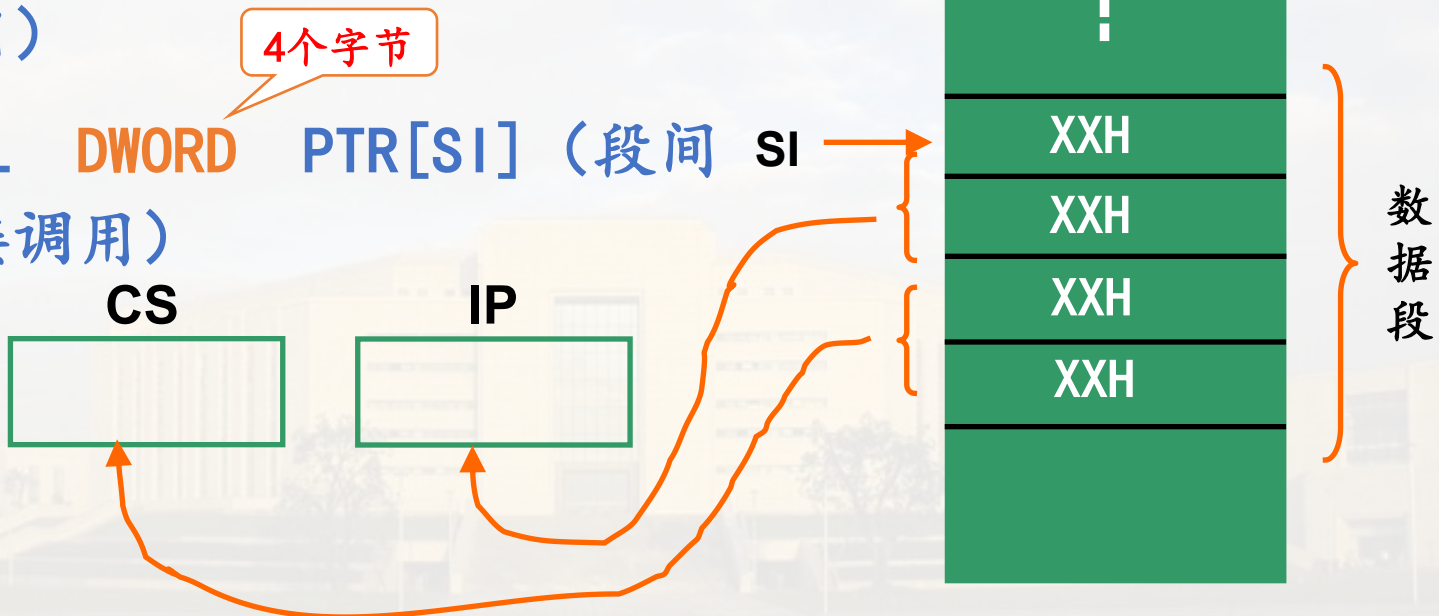
- 格式:

✓ **CALL FAR PROC**

- 格式例:

✓ **CALL FAR TIMER** (段间直接调用)

✓ **CALL DWORD PTR[SI]** (段间间接调用)

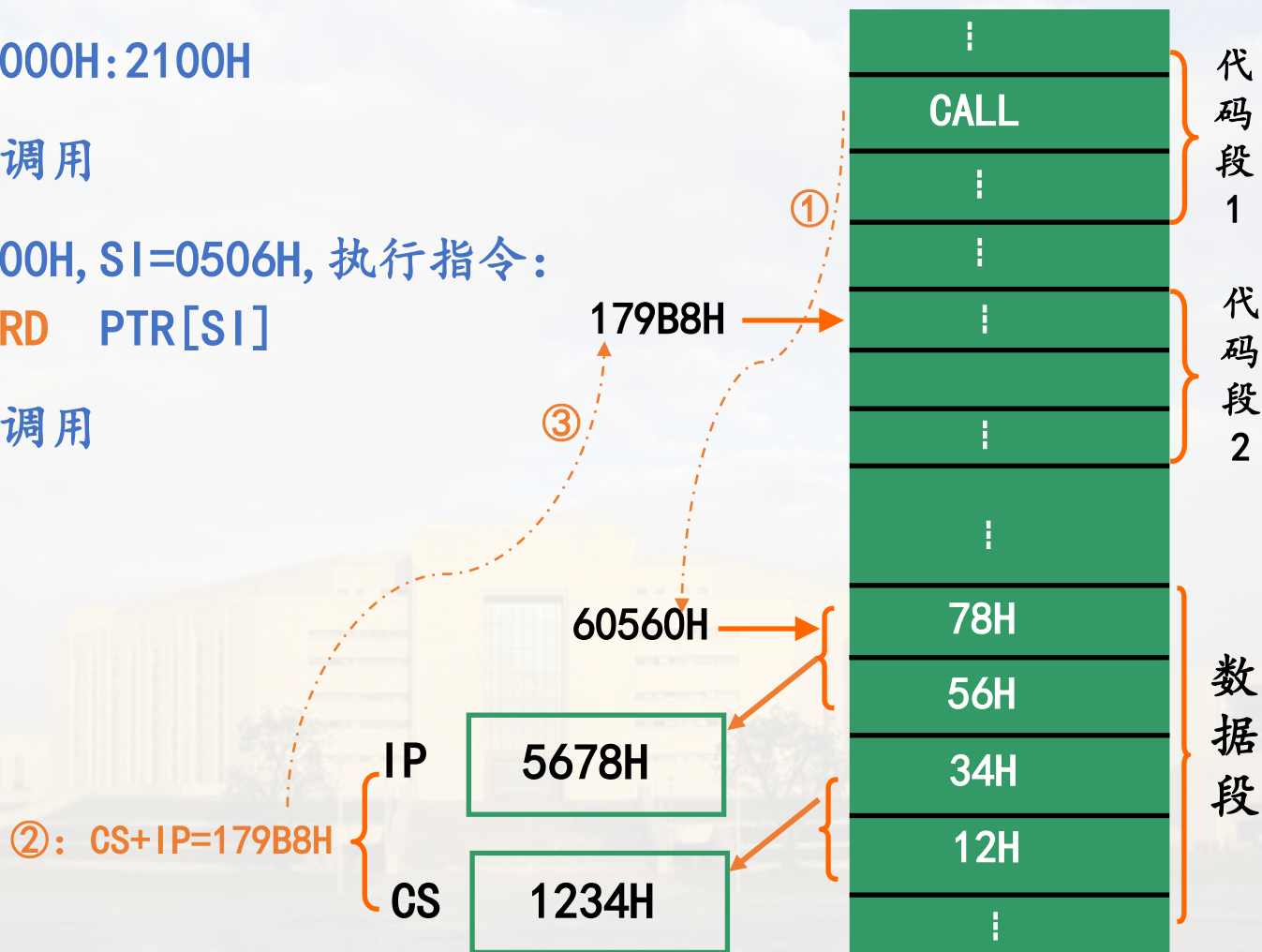


3.6 程序控制指令

段间调用例

● 例：

- ✓ 1. `CALL 3000H:2100H`
- ✓ ;段间直接调用
- ✓ 2. 设 `DS=6000H`, `SI=0506H`, 执行指令：
`CALL DWORD PTR[SI]`
- ✓ ;段间间接调用



» 返回指令



功能

从堆栈中弹出断点地址，返回原程序



格式

RET

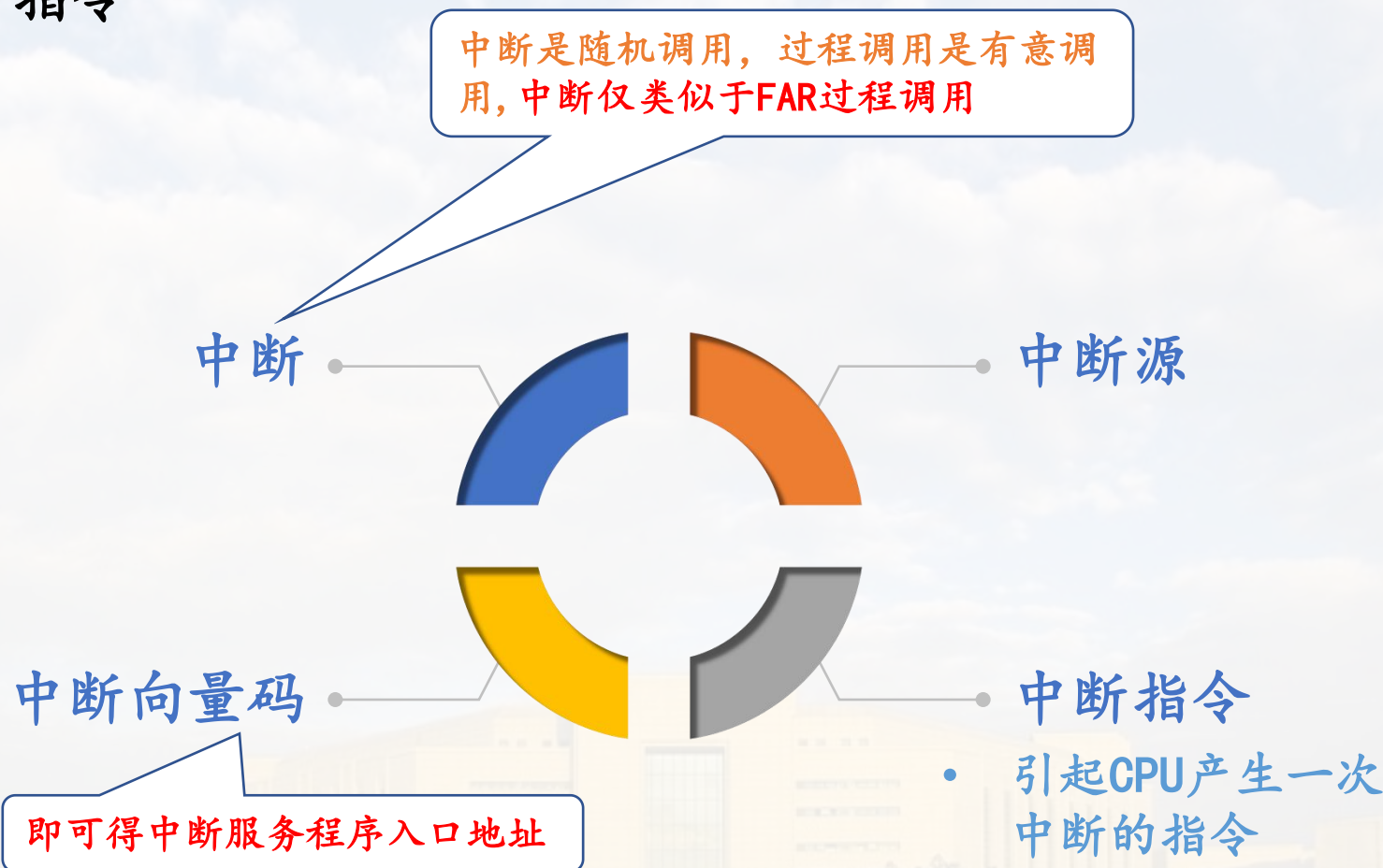
返回 { IP: 段内调用
CS+IP: 段间调用



说明

- RET指令一般位于子程序的最后。
- 返回指令在格式上不区分段内或段间（即1WORD或2WORD）。

» 中断指令



3.6 程序控制指令

» 中断与过程调用



中断是随机事件或异常事件引起，过程调用则是事先已在程序中安排好；



响应中断请求不仅要保护断点地址，还要保护FLAGS内容；（CALL调用不需要）



过程调用指令在指令中直接给出子程序入口地址，中断指令只给出中断向量码，入口地址则在向量码指向的内存单元中（即中断向量表中）。

中断与过程调用

3.6 程序控制指令

中断指令（软指令）

- 格式：

INT n

中断类型码：n=0 ~ 255

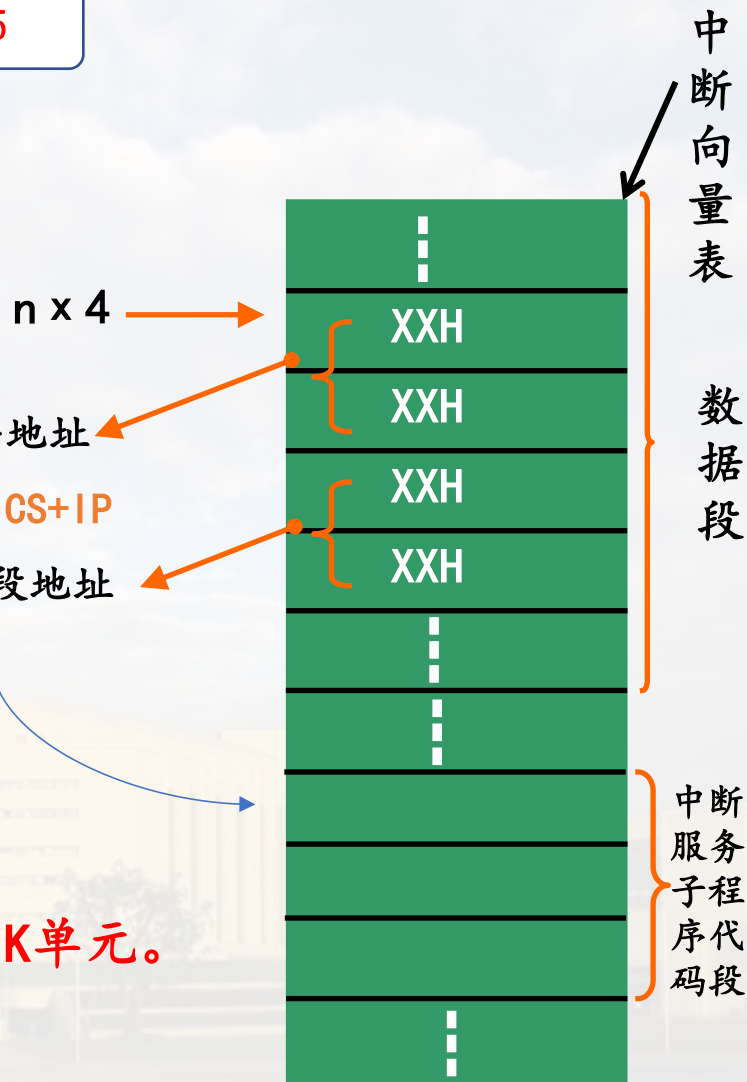
- 说明：

$n \times 4$

存放“中断服务子程序入口地址”
的存储单元的偏移地址

该单元在数据段，段地址：DS

说明：8086的中断向量表存放在内存最低的1K单元。



» 中断指令的执行过程

01

将FLAGS压入堆栈；（保护硬现场）

02

将INT指令的下一条指令的CS、IP压栈；（断点）

03

由 $n \times 4$ 得到存放中断向量的地址；（访问中断向量表）

04

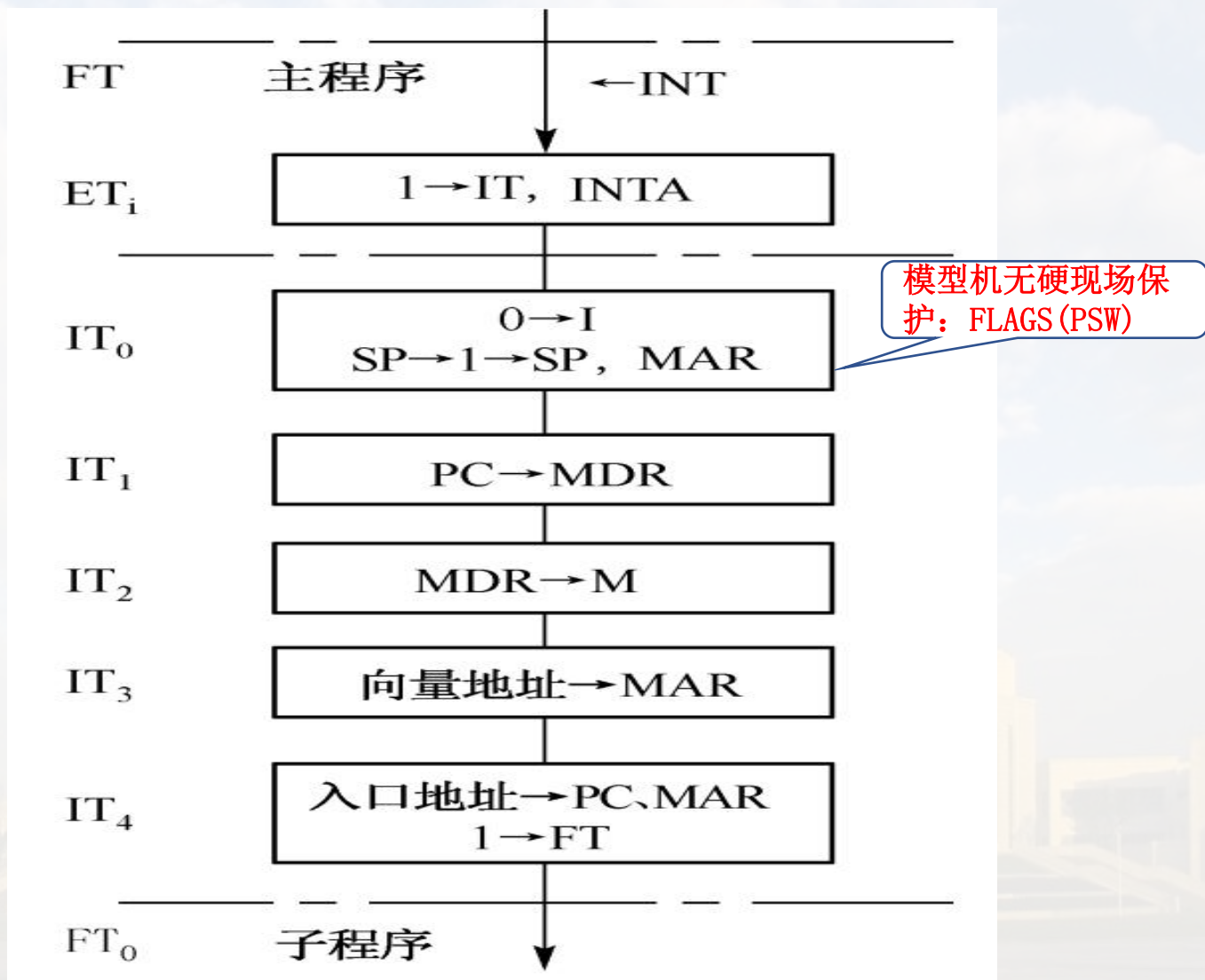
将中断向量（中断服务程序入口地址）送CS和IP寄存器；

05

转入中断服务程序。

3.6 程序控制指令

中断周期IT:

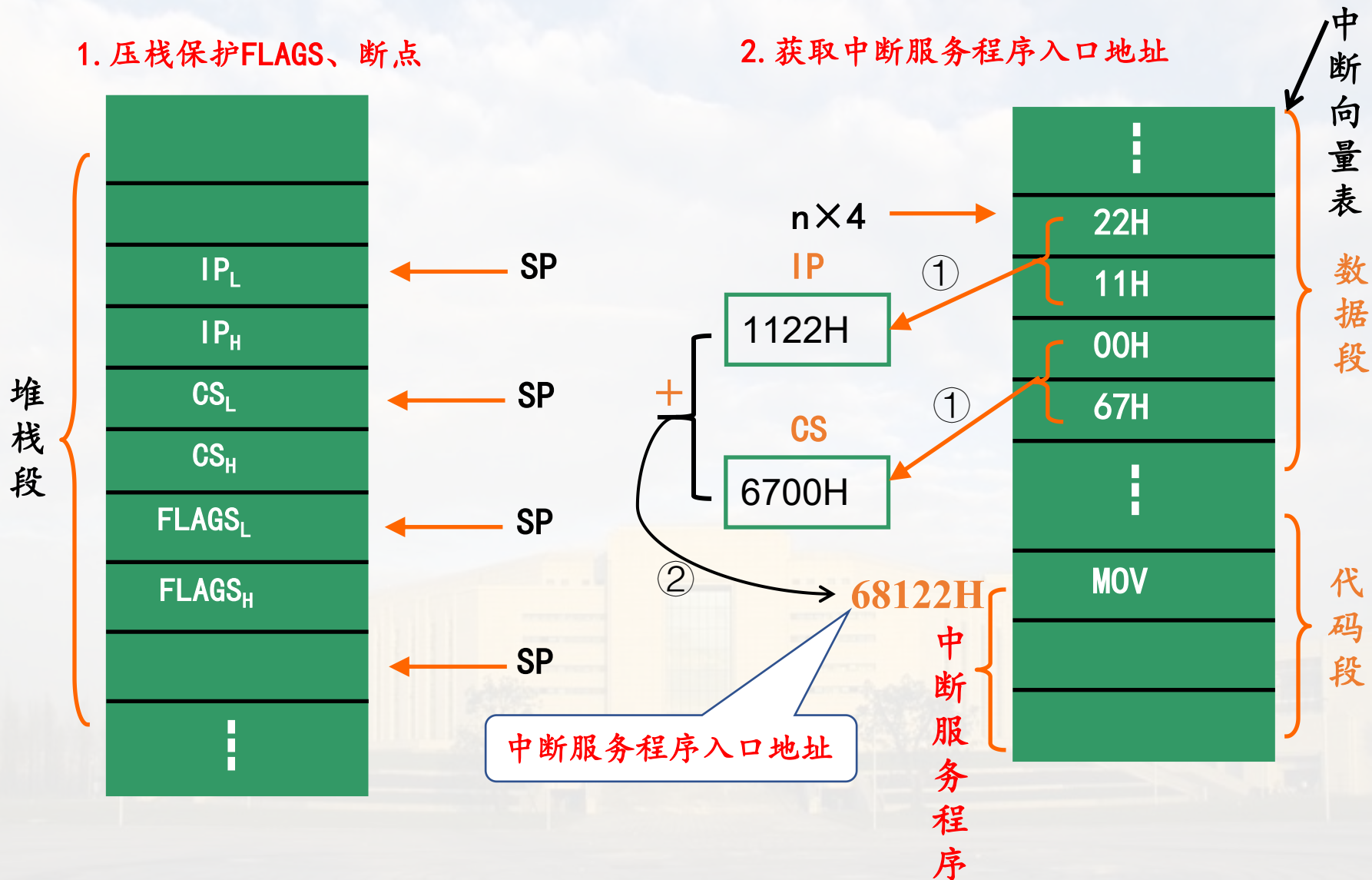


3.6 程序控制指令

中断指令的执行过程

1. 压栈保护FLAGS、断点

2. 获取中断服务程序入口地址



3.6 程序控制指令

中断指令例

执行程序段：

CS	IP	
		⋮
6200H:0110H		INT 21H
6200H:0112H		MOV AX, BX
		⋮

压栈保护FLAGS、断点

执行INT指令后的
压栈操作

SP=11FA

硬现场：FLAGS

SP=1200H

SP初始值

11FCH

11FEH

12H

01H

00H

62H

FLAGSL

FLAGSH

断点

堆栈段

3.6 程序控制指令

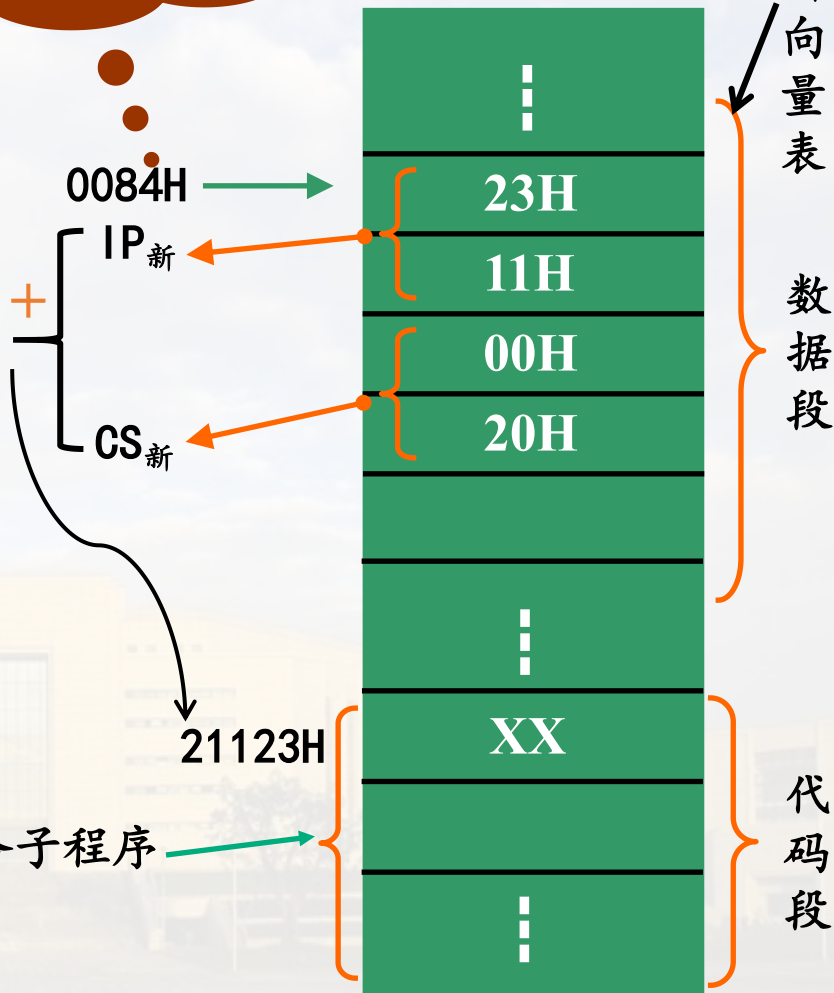
中断指令例

■ 执行INT 21H指令后

$IP = [21H \times 4] ; 84H$

$CS = [(21H \times 4) + 2] ; 86H$

执行INT指令后的
获取入口地址操作



» 溢出中断指令



格式

INTO → 相当于 INT 4

指令执行时，检查OF标志
OF=1，则INT 4，类似一条转移指令
OF=0，则顺序执行下一条指令



指令执行时检查OF标志：

- 若OF=1，则启动一个类型为4的中断过程，给出一个出错标志，若OF=0，不做任何操作。



说明

- INTO指令通常安排在有符号数加减运算指令之后，判断是否发生溢出，即同号相加或异号相减。

» 中断返回指令



格式

IRET

编程实现，即IRET指令实现

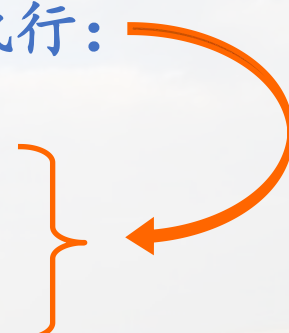


说明

- 中断服务程序的最后一条指令，执行：

恢复断点

恢复标志寄存器内容



但现场的恢复要使用POP指令

» 处理器控制指令



对标志位的操作

与外部设备的同步



说明见教材
P145表3-5

» 本章课外自学内容

自学
内容

01

BCD码调整指令

02

条件循环指令LOOPZ, LOOPNZ

03

Pentium新增指令

» 结束语

掌握

01

指令的格式及意义；

02

指令对操作数的要求及对标志位的影响；

03

指令的应用。

» 作业

- 3.2 (4)、(6)、(7)、(8)
- 3.3 (2)、(3)
- 3.9
- 3.11 (2)
- 3.15

THANK

@刘辉

